

NORTHWESTERN UNIVERSITY

Generalized t-SNE and Beyond: Probabilistic Methods for Dimensionality
Reduction, Combinatorial Optimization, and Machine Learning

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

DOCTOR OF PHILOSOPHY

Field of Mathematics

By

Yi Gu

EVANSTON, ILLINOIS

June 2025

© Copyright by Yi Gu 2025

All Rights Reserved

Abstract

Generalized t-SNE and Beyond: Probabilistic Methods for Dimensionality Reduction,
Combinatorial Optimization, and Machine Learning

Yi Gu

This is the abstract.

Acknowledgements

Text for acknowledgments.

Preface

This is the preface.

Dedication

This thesis is dedicated to my parents and all the people who have supported me and helped me along the way. I love you all.

“τὰ καλὰ χαλεπά (The beautiful things are difficult).”

- Plato

Table of Contents

Abstract	3
Acknowledgements	4
Preface	5
Dedication	6
Table of Contents	7
List of Tables	9
List of Figures	11
Chapter 1. Convergence of t-Distributed Stochastic Neighbor Embedding with Generalized Input and Output Kernels	14
1.1. First section of chapter 1	14
1.2. Another section	14
Chapter 2. Learning Markov Random Fields for Combinatorial Structures via Sampling through Lovász Local Lemma	15
Overview	15
2.1. Introduction	16
2.2. Preliminaries	19

	8
2.3. Sampling Through Lovász Local Lemma	23
2.4. Neural Lovász Sampler	25
2.5. Experiments	30
2.6. Chapter Recap	37
2.7. Probability Distribution of Algorithm 1	38
2.8. Running Time Analysis of Algorithm 1	49
2.9. Constrained MRF Model	56
2.10. Experiment Settings and Configurations	58
Chapter 3. VarFA: A Variational Factor Analysis Framework for Efficient Bayesian	
Learning Analytics	71
Overview	71
3.1. Introduction	72
3.2. Background and Related Work	76
3.3. VarFA : A Variational Inference Factor Analysis Framework	83
3.4. Experiments	89
3.5. Chapter Conclusion and Future Work	99
Bibliography	102

List of Tables

2.1	Sampling efficiency and accuracy for learning K -SAT solutions with preferences. The proposed NELSON is the most efficient (see “Training Time Per Epoch”) and always generates valid assignments (see “Validness”) with a small approximation error (see “Approximation Error of Gradient”) against all baselines. T.O. means time out.	32
2.2	The quality of learning outcomes for learning random K -SAT solutions with preferences. NELSON achieves the best likelihood and MAP@10 scores. T.O. is time out.	33
2.3	Sample efficiency and learning performance of the sink-free orientation task. NELSON is the most efficient (see Training Time Per Epoch) and always generates valid assignments (see Validness), has the smallest error approximating gradients, and has the best learning performance (see MAP@10) among all baselines.	36
2.4	Summary of all the notations used in the theoretical analysis of Algorithm 1.	41
2.5	4 constraints for converting pairwise terms in the potential function into single variable form.	57
3.1	Summary statistics of pre-processed real data sets.	92

- 3.2 Student answer prediction performance comparing VarFA to SPARFA-M on Assistment, Algebra and Bridge data sets. $\uparrow$ and $\downarrow$ denote higher and lower is better, respectively. VarFA performs better than SPARFA-M on all three data sets and evaluation metrics most of the time. Additionally, VarFA's run time is very close to SPARFA-M. 94
- 3.3 Illustration of the estimated latent skills with the their top 3 most strongly associated skill tags in the Assistment data set. The percentage in the parenthesis shows the association probability (summed to 1 for each latent skill). We see that the tagged skills associated with each estimated latent skill form intuitive and interpretable groups. 97

List of Figures

- 2.1 Running time and the percentage of valid structures sampled uniformly at random from solutions of K-SAT problems. Among all the problem sizes, NELSON always generate valid solutions and is the most efficient sampler. 34
- 2.2 Running time, the percentage of valid solutions generated, and rounds of resampling for weighted sample generation of K-SAT solutions. Among all the problem sizes, NELSON scales the best among all approaches and always generates valid solutions. 34
- 2.3 Frequency histograms for the number of resample and the total time of NELSON method for uniformly sampling visiting paths for vehicle routing problem. 38
- 2.4 Implementation pipeline of the NELSON-CD algorithm with $m = 1$. The proposed NELSON can be efficiently adapted to a Pytorch-based machine learning library and enforces constraint satisfaction during learning. 61
- 2.5 **(a)** An un-directed graph $G(V, E)$ where the vertices are $V = \{v_1, v_2, v_3, v_4\}$ and the un-directed edges are $E = \{e_1 = (v_1, v_2), e_2 = (v_1, v_3), e_3 = (v_2, v_3), e_4 = (v_2, v_4), e_5 = (v_3, v_4)\}$. **(b)** A possible sink-free

orientation of the edges in the graph and its matrix representation x ,
 where every vertex has at least one outgoing edge. 64

2.6 The distribution of resampling steps in the NELSON and Algorithmic-
 LLL [Moser and Tardos, 2010]. Both of them get a valid sample within
 $T_{tryouts}$. NELSON takes much fewer resamples than Algorithmic-LLL
 because it resamples all the violated clauses at every iteration while
 Algorithmic-LLL only resamples one of them. 70

3.1 Performance of the SPARFA-M, SPARFA-B, and VarFA algorithms on
 the synthetic data set with different data sizes. Plots from left to right
 show comparison on accuracy (ACC), area under curve (AUC) and F1
 metrics, respectively. Higher is better for all metrics. VarFA performs
 similarly to SPARFA-M and SPARFA-B. 88

3.2 Training run time comparison among VarFA, SPARFA-M, and
 SPARFA-B on synthetic data sets of varying sizes. VarFA performs
 approximate Bayesian inference almost 100x faster than SPARFA-B and
 is close to the run time of SPARFA-M. Thus, VarFA enables practical
 and scalable Bayesian inference for very large data sets. 91

3.3 Violin plot showing the mean and standard deviation of the estimated
 skill mastery levels on 10 selected students on the 3rd, 4th and 7th
 latent skills that VarFA computes. In each sub-figure, bottom and top
 axes respectively shows student IDs and top axis shows the number

of questions each student answered. The more questions a student answers, the tighter the credible interval. (Best viewed in color.) 93

3.4 Comparison between the estimated skill mastery levels using VarFA's predictions and using empirical observations for student with ID 110. Even though the two curves show different numeric values, they nevertheless demonstrate similar trends, showing that the predictions reasonably match our intuition about student's skill mastery levels. 101

CHAPTER 1

Convergence of t-Distributed Stochastic Neighbor Embedding with Generalized Input and Output Kernels

... text ...

1.1. First section of chapter 1

... text ...

1.2. Another section

... text ...

CHAPTER 2

Learning Markov Random Fields for Combinatorial Structures via Sampling through Lovász Local Lemma

Overview

Learning to generate complex combinatorial structures satisfying constraints will have transformative impacts in many applications. However, it is beyond the capabilities of existing approaches due to the highly intractable nature of the embedded probabilistic inference. Prior works spend most of the training time learning to separate valid from invalid structures but not valid structures inside and outside of the training set. We develop NEural Lovász Sampler (NELSON), which embeds the sampler through Lovász Local Lemma (LLL) as a fully differentiable neural network layer. The NELSON sampler allows us to obtain valid samples from the current model distribution. Contrastive divergence is then applied to separate these samples from those in the training set. NELSON is implemented as a fully differentiable neural net, taking advantage of the parallelism of GPUs. Experimental results on several real-world domains reveal that NELSON learns to generate 100% valid structures, while baselines either time out or cannot ensure validity. NELSON also outperforms other approaches in running time, log-likelihood, and MAP scores.

2.1. Introduction

In recent years, tremendous progress has been made in generative modeling [Hinton, 2002; Tsochantaridis et al., 2005; Goodfellow et al., 2014; Kingma and Welling, 2014; Germain et al., 2015; Larochelle and Murray, 2011; van den Oord et al., 2016; Arjovsky et al., 2017; Song and Ermon, 2019; Song et al., 2021; Murphy et al., 1999; Yedidia et al., 2000; Wainwright and Jordan, 2006]. In the meantime, tremendous progress has been made in automated reasoning [Braunstein et al., 2005; Andersen et al., 2007; Chavira et al., 2006; Van Hentenryck, 1989; Gogate and Dechter, 2012; Sang et al., 2005]. Nevertheless, reasoning and learning have been growing independently for a long time. Only recently do ideas emerge exploring the role of reasoning in learning [Kusner et al., 2017; Jin et al., 2018; Dai et al., 2018; Hu et al., 2017; Lowd and Domingos, 2008; Ding et al., 2021].

Learning a generative model involves increasing the divergence in likelihood scores between the structures in the training set and those structures sampled from the current generative model distribution. While current approaches have achieved successes in *un-structured* domains such as vision or speech, their performance is degraded in the structured domain, because it is already computationally intractable to search for a valid structure in a combinatorial space subject to constraints, not to mention sampling, which has a higher complexity. In fact, when applied in a constrained domain, existing approaches spend most of their training time manipulating the likelihood of invalid structures, but not learning the difference between valid structures inside and outside of the training set.

The Lovász Local Lemma (LLL) [Erdős and Lovász, 1973] is a classic gem in combinatorics, which states that there exists a positive probability that none of *bad* events occur,

as long as these events are *mostly* independent from one another and are not too likely individually. Recently, [Moser and Tardos, 2010] came up with an algorithm (noted as algorithmic-LLL), in which find such a instance by executing the algorithm. [Guo et al., 2019] further proved that the algorithmic-LLL is an unbiased sampler if those bad events satisfy the so-called “extreme” condition. The expected running time of the sampler is also shown to be polynomial. As one contribution of this piece of research, we offer proofs of the two aforementioned results using precise mathematical notations, clarifying a few descriptions not precisely defined in the original proof. While this line of research clearly demonstrates the potential of LLL in generative learning (generating samples that satisfy all hard constraints), it is not clear how to embed LLL-based samplers into learning and no empirical studies have been performed to evaluate LLL-based samplers in machine learning.

In this chapter, we develop NEural Lovász Sampler (NELSON), which implements the LLL-based sampler as a fully differentiable neural network module. Our NELSON-CD embeds NELSON into the contrastive divergence learning process of Markov Random Fields (MRFs). Embedding an LLL-based sampler allows the contrastive learning algorithm to focus on learning the difference between the training data and the valid structures drawn from the current model distribution. Baseline approaches, on the other hand, spend most of their training time learning to generate valid structures. In addition, NELSON is fully differentiable, hence allowing for efficient learning harnessing the parallelism of GPUs.

We conduct experiments NELSON-CD on learning preferences towards (i) random K -satisfiability solutions (ii) sink-free orientations of un-directed graphs and (iii) vehicle delivery routes. In all these applications, NELSON-CD (i) has the fastest training time due

to seamless integration into the learning framework (shown in Tables 2.1(a), 2.3(a)). (ii) NELSON generates samples 100% satisfying constraints (shown in Tables 2.1(b), 2.3(b)), which facilitates effective contrastive divergence learning. Other baselines either cannot satisfy constraints or time out. (iii) The fast and valid sample generation allows NELSON to obtain the best learning performance (shown in Table 2.1(c), 2.2(a,b), 2.3(c,d)).

Our contributions can be summarized as follows: **(a)** We present NELSON-CD, a contrastive divergence learning algorithm for constrained MRFs driven by sampling through the Lovász Local Lemma (LLL). **(b)** Our LLL-based sampler (NELSON) is implemented as a fully differentiable multi-layer neural net, allowing for end-to-end training on GPUs. **(c)** We offer a mathematically sound proof of the sample distribution and the expected running time of the NELSON algorithm. **(d)** Experimental results reveal the effectiveness of NELSON in (i) learning models with high likelihoods (ii) generating samples 100% satisfying constraints and (iii) having high efficiency in training¹.

2.1.1. Related Work

Related to our NELSON are neural-based approaches to solve combinatorial optimization problems [Selsam et al., 2019; Duan et al., 2022; Li et al., 2018]. Machine learning is also used to discover better heuristics [Yolcu and Póczos, 2019; Chen and Tian, 2019]. Reinforcement learning [Karalias and Loukas, 2020; Bengio et al., 2021] as well as approaches integrating search with neural nets [Mandi et al., 2020] are found to be effective in solving combinatorial optimization problems as well. Regarding probabilistic inference, there are a rich line of research on MCMC-type sampling [Neal, 1993; Dagum and Chavez, 1993; Ge

¹Code is at: <https://github.com/jiangnanhugo/nelson-cd>

et al., 2018] and various versions of belief propagation [Murphy et al., 1999; Ihler et al., 2005; Coja-Oghlan et al., 2020; Ding and Xue, 2020]. SampleSearch [Gogate and Dechter, 2011] integrates importance sampling with constraint-driven search. Probabilistic inference based on hashing and randomization obtains probabilistic guarantees for marginal queries and sampling via querying optimization oracles subject to randomized constraints [Gomes et al., 2006; Ermon et al., 2013; Achlioptas and Theodoropoulos, 2017; Chakraborty et al., 2013].

2.2. Preliminaries

2.2.1. Markov Random Fields (MRF)

represent a Boltzmann distribution of the discrete variables $X = \{X_i\}_{i=1}^n$ over a Boolean hypercube $\mathcal{X} = \{0, 1\}^n$. For $x \in \mathcal{X}$, we have:

$$(2.1) \quad P_\theta(X = x) = \frac{\exp(\phi_\theta(x))}{Z(\theta)}.$$

Here, $Z(\theta) = \sum_{x' \in \mathcal{X}} \exp(\phi_\theta(x'))$ is the partition function that normalizes the total probability to 1. The potential function is $\phi_\theta(x) = \sum_{j=1}^m \phi_{\theta,j}(x_j)$. Each $\phi_{\theta,j}$ is a *factor potential*, which maps a value assignment over a subset of variables $X_j \subseteq X$ to a real number. We use upper case letters, such as X_j to represent (a set of) random variables, and use lower case letters, such as x_j , to represent its value assignment. We also use $\text{var}(\phi_{\theta,j})$ to represent the domain of $\phi_{\theta,j}$, i.e., $\text{var}(\phi_{\theta,j}) = X_j$. θ are the parameters to learn.

2.2.2. Constrained MRF

is the MRF model subject to a set of hard constraints $\mathcal{C} = \{c_j\}_{j=1}^L$. Here, each constraint c_j limits the value assignments of a subset of variables $\text{var}(c_j) \subseteq X$. We write $c_j(x) = 1$ if the assignment x satisfies the constraint c_j and 0 otherwise. Note that x is an assignment to all random variables, but c_j only depends on variables $\text{var}(c_j)$. We denote $C(x) = \prod_{j=1}^L c_j(x)$ as the indicator function. Clearly, $C(x) = 1$ if all constraints are satisfied and 0 otherwise. The constrained MRF is:

$$(2.2) \quad P_\theta(X = x|\mathcal{C}) = \frac{\exp(\phi_\theta(x)) C(x)}{Z_{\mathcal{C}}(\theta)},$$

where $Z_{\mathcal{C}}(\theta) = \sum_{x' \in \mathcal{X}} \exp(\phi_\theta(x')) C(x')$ sums over only valid variable assignments.

2.2.3. Learn Constrained MRF

Given a data set $\mathcal{D} = \{x^k\}_{k=1}^N$, where each x^k is a valid assignment that satisfies all constraints, learning can be achieved via maximal likelihood estimation. The optimal parameters θ^* is obtained by minimizing the negative log-likelihood $\ell_{\mathcal{C}}(\theta)$:

$$(2.3) \quad \begin{aligned} \ell_{\mathcal{C}}(\theta) &= -\frac{1}{N} \sum_{k=1}^N \log P_\theta(X = x^k|\mathcal{C}) \\ &= -\frac{1}{N} \sum_{k=1}^N \phi_\theta(x^k) + \log Z_{\mathcal{C}}(\theta). \end{aligned}$$

The parameters θ can be trained using gradient-based optimization: $\theta^{t+1} = \theta^t - \eta \nabla \ell_{\mathcal{C}}(\theta)$, where η is the learning rate. Let $\nabla \ell_{\mathcal{C}}(\theta)$ denotes the gradient of the objective $\ell_{\mathcal{C}}(\theta)$, that

is calculated as:

$$\begin{aligned}
 (2.4) \quad \nabla \ell_{\mathcal{C}}(\theta) &= -\frac{1}{N} \sum_{k=1}^N \nabla \phi_{\theta}(x^k) + \nabla \log Z_{\mathcal{C}}(\theta) \\
 &= -\mathbb{E}_{x \sim \mathcal{D}} (\nabla \phi_{\theta}(x)) + \mathbb{E}_{\tilde{x} \sim P_{\theta}(x|\mathcal{C})} (\nabla \phi_{\theta}(\tilde{x})).
 \end{aligned}$$

The first term is the expectation over all data in training set $\mathcal{D}$. During training, this is approximated using a mini-batch of data randomly drawn from the training set $\mathcal{D}$. The second term is the expectation over the current model distribution $P_{\theta}(X = x|\mathcal{C})$ (detailed in section 2.9.2). Because learning is achieved following the directions given by the divergence of two expectations, this type of learning is commonly known as contrastive divergence (CD) [Hinton, 2002]. Estimating the second expectation is the bottleneck of training because it is computationally intractable to sample from this distribution subject to combinatorial constraints. Our approach, NELSON, leverages the sampling through Lovász Local Lemma to approximate the second term.

2.2.4. Factor Potential in Single Variable Form

Our method requires each factor potential $\phi_{\theta,j}(x_j)$ in Eq. (2.1) to involve only one variable. This is NOT an issue as *all constrained MRF models can be re-written in single variable form* by introducing additional variables and constraints. Our transformation follows the idea in [Sang et al., 2005]. We illustrate the idea by transforming one-factor potential $\phi_{\theta,j}(x_j)$ into the single variable form. First, notice all functions including $\phi_{\theta,j}(x_j)$ over a

Boolean hypercube $\{0, 1\}^n$ have a (unique) discrete Fourier expansion:

$$(2.5) \quad \phi_{\theta,j}(x_j) = \sum_{S \in [\mathbf{var}(\phi_{\theta,j})]} \hat{\phi}_{\theta,j,S} \chi_S(x).$$

Here, $\chi_S(x) = \prod_{X_i \in S} X_i$ is the basis function and $\hat{\phi}_{\theta,j,S}$ are Fourier coefficients. $[\mathbf{var}(\phi_{\theta,j})]$ denotes the power set of $\mathbf{var}(\phi_{\theta,j})$. For example, if $\mathbf{var}(\phi_{\theta,j}) = \{X_1, X_2\}$, then we have $[\mathbf{var}(\phi_{\theta,j})] = \{\emptyset, \{X_1\}, \{X_2\}, \{X_1, X_2\}\}$. See [Mansour, 1994] for details of Fourier transformation. To transform $\phi_{\theta,j}(x_j)$ into single variable form, we introduce a new Boolean variable $\hat{\chi}_S$ for every $\chi_S(x)$. Because $\hat{\chi}_S$ and all X_i 's are Boolean, we can use combinatorial constraints to guarantee $\hat{\chi}_S = \prod_{X_i \in S} X_i$. These constraints are incorporated into $\mathcal{C}$. Afterward, $\phi_{\theta,j}(x_j)$ is represented as the sum of several single-variable factors. Notice this transformation is only possible when the MRF is subject to constraints. We offer a detailed example in section 2.9.1 for further explanation. Equipped with this transformation, we assume all $\phi_{\theta,j}(x_j)$ are single variable factors for the rest of the chapter.

2.2.5. Extreme Condition

The set of constraints $\mathcal{C}$ is called “extremal” if no variable assignment violates two constraints sharing variables, according to [Guo et al., 2019].

Condition 1. *A set of constraints $\mathcal{C}$ is called extremal if and only if for each pair of constraints $c_i, c_j \in \mathcal{C}$, (i) either their domain variables do not intersect, i.e., $\mathbf{var}(c_i) \cap \mathbf{var}(c_j) = \emptyset$. (ii) or for all $x \in \mathcal{X}$, $c_i(x) = 1$ or $c_j(x) = 1$.*

2.3. Sampling Through Lovász Local Lemma

Lovász Local Lemma (LLL) [Erdős and Lovász, 1973] is a fundamental method in combinatorics to show the existence of a valid instance that avoids all the bad events, if the occurrences of these events are “mostly” independent and are not very likely to happen individually. Since the occurrence of a bad event is equivalent to the violation of a constraint, we can use the LLL-based sampler to sample from the space of constrained MRFs. To illustrate the idea of LLL-based sampling, we assume the constrained MRF model is given in the single variable form (as discussed in the previous section):

$$(2.6) \quad P_\theta(X = x|\mathcal{C}) = \frac{\exp(\sum_{i=1}^n \theta_i x_i) C(x)}{Z_{\mathcal{C}}(\theta)},$$

where $Z_{\mathcal{C}}(\theta) = \sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) C(x')$.

As shown in Algorithm 1, the LLL-based sampler [Guo et al., 2019] takes the random variables $X = \{X_i\}_{i=1}^n$, the parameters of constrained MRF θ , and constraints $\mathcal{C} = \{c_j\}_{j=1}^L$ that satisfy Condition 1 as the inputs. In Line 1 of Algorithm 1, the sampler gives an initial random assignment of each variable following its marginal probability: $x_i \sim \frac{\exp(\theta_i x_i)}{\sum_{x_i \in \{0,1\}} \exp(\theta_i x_i)}$, for $1 \leq i \leq n$. Here we mean that x_i is chosen with probability mass $\frac{\exp(\theta_i x_i)}{\sum_{x_i \in \{0,1\}} \exp(\theta_i x_i)}$. Line 2 of Algorithm 1 checks if the current assignment satisfies all constraints in $\mathcal{C}$. If so, the algorithm terminates. Otherwise, the algorithm finds the set of violated constraints $S = \{c_j | c_j(x) = 0, c_j \in \mathcal{C}\}$ and re-samples related variables $X_k \in \text{var}(S)$ using the same marginal probability, *i.e.*, $x_k \sim \frac{\exp(\theta_k x_k)}{\sum_{x_k \in \{0,1\}} \exp(\theta_k x_k)}$. Here $\text{var}(S) = \cup_{c_j \in S} \text{var}(c_j)$. The algorithm repeatedly samples all those random variables violating constraints until all the constraints are satisfied.

Algorithm 1 Sampling Through Lovász Local Lemma.

Input: Random variables $X = \{X_i\}_{i=1}^n$; Constraints $\mathcal{C} = \{c_j\}_{j=1}^L$; Parameters of the constrained MRF θ .

- 1: $x_i \sim \frac{\exp(\theta_i x_i)}{\sum_{x_i \in \{0,1\}} \exp(\theta_i x_i)}$, for $1 \leq i \leq n$. $\triangleright$ initialize
 - 2: **while** $C(x) = 0$ **do**
 - 3: Find all violated constraints $S \subseteq \mathcal{C}$ in x .
 - 4: $x_k \sim \frac{\exp(\theta_k x_k)}{\sum_{x_k \in \{0,1\}} \exp(\theta_k x_k)}$, for $x_k \in \text{var}(S)$. $\triangleright$ resample
 - 5: **end while return** A valid sample x drawn from $P_\theta(X = x|\mathcal{C})$.
-

Under Condition 1, Algorithm 1 guarantees each sample is from the constrained MRFs' distribution $P_\theta(X = x|\mathcal{C})$ (shown in Theorem 1). In section 2.7, we present the detailed proof and clarify the difference to the original descriptive proof [Guo et al., 2019].

Theorem 1 (Probability Distribution). *Given random variables $X = \{X_i\}_{i=1}^n$, constraints $\mathcal{C} = \{c_j\}_{j=1}^L$ that satisfy Condition 1, and the parameters of the constrained MRF in the single variable form θ . Upon termination, Algorithm 1 outputs an assignment x that is randomly drawn from the constrained MRF distribution: $x \sim P_\theta(X = x|\mathcal{C})$.*

SKETCH OF PROOF. We first show that in the last round, the probability of obtaining two possible assignments conditioning on all previous rounds in Algorithm 1 has the same ratio as the probability of those two assignments under distribution $P_\theta(X = x|\mathcal{C})$. Then we show when Algorithm 1 ends, the set of all possible outputs is equal to the domain of non-zero probabilities of $P_\theta(X = x|\mathcal{C})$. Thus we conclude the execution of Algorithm 1 produces a sample from $P_\theta(X = x|\mathcal{C})$ because of the identical domain and the match of probability ratios of any two valid assignments. $\square$

The expected running time of Algorithm 1 is determined by the number of rounds of re-sampling. In the uniform case that $\theta_1 = \dots = \theta_n$, the running time is linear in the size

of the constraints $\mathcal{O}(L)$. The running time for the weighted case has a closed form. We leave the details in section 2.8.

2.4. Neural Lovász Sampler

We first present the proposed Neural Lovász Sampler (NELSON) that implements the LLL-based sampler as a neural network, allowing us to draw multiple samples in parallel on GPUs. We then demonstrate how NELSON is embedded in CD-based learning for constrained MRFs.

2.4.1. NELSON: Neural Lovász Sampler

Represent Constraints as CNF. NELSON obtains samples from the constrained MRF model in single variable form (Eq. 2.6). To simplify notations, we denote $P_\theta(X_i = x_i) = \frac{\exp(\theta_i x_i)}{\sum_{x_i \in \{0,1\}} \exp(\theta_i x_i)}$. Since our constrained MRF model is defined on the Boolean hyper-cube $\{0,1\}^n$, we assume all constraints $\{c_j\}_{j=1}^L$ are given in the Conjunctive Normal Form (CNF). Note that all propositional logic can be reformulated in CNF format with at most a polynomial-size increase. A formula represented in CNF is a conjunction ($\wedge$) of clauses. A clause is a disjunction ($\vee$) of literals, and a literal is either a variable or its negation ($\neg$). Mathematically, we use c_j to denote a clause and use $l_{j,k}$ to denote a literal. In this case, a CNF formula would be:

$$(2.7) \quad c_1 \wedge \dots \wedge c_L, \quad \text{where } c_j = l_{j,1} \vee \dots \vee l_{j,K}$$

A clause is true if and only if at least one of the literals in the clause is true. The whole CNF is true if all clauses are true.

We transform each step of Algorithm 1 into arithmetic operations, hence encoding it as a multi-layer neural network. To do that, we first need to define a few notations:

- Vector of assignment $x^t = (x_1^t, \dots, x_n^t)$, where x_i^t is the assignment of variable X_i in the t -th round of Algorithm 1. $x_i^t = 1$ denotes variable X_i takes value 1 (or true).
- Vector of marginal probabilities $P = (P_1, \dots, P_n)$, where P_i is the probability of variable X_i taking value 0 (false): $P_i = P_\theta(X_i = 0) = \exp(0)/(\exp(0) + \exp(\theta_i))$.
- Tensor $W \in \{-1, 0, 1\}^{L \times K \times n}$ and matrix $b \in \{0, 1\}^{L \times n}$, that are used for checking constraint satisfaction:

$$(2.8) \quad W_{jki} = \begin{cases} 1 & \text{if } k\text{-th literal of clause } c_j \text{ is } X_i, \\ -1 & \text{if } k\text{-th literal of clause } c_j \text{ is } \neg X_i, \\ 0 & \text{otherwise.} \end{cases}$$

$$(2.9) \quad b_{jk} = \begin{cases} 1 & \text{if } k\text{-th literal of clause } c_j \text{ is negated,} \\ 0 & \text{otherwise.} \end{cases}$$

- Matrix $V \in \{0, 1\}^{L \times n}$, denoting the mapping from clauses to variables in the CNF form for constraints $\mathcal{C}$:

$$(2.10) \quad V_{ji} = \begin{cases} 1 & \text{if clause } c_j \text{ contains a literal involving } X_i \\ 0 & \text{otherwise.} \end{cases}$$

- Vector of resampling indicators A^t , where $A_i^t = 1$ indicates variable X_i needs to be resampled at round t .

Given these defined variables, we represent each step of Algorithm 1 using arithmetic operations as follows:

Initialization. To complete line 1 of Algorithm 1, given the marginal probability vector P , the first step is sampling an initial assignment of X , $x^1 = (x_1^1, \dots, x_n^1)$. It is accomplished by: for $1 \leq i \leq n$,

$$(2.11) \quad x_i^1 = \begin{cases} 1 & \text{if } u_i > P_i, \\ 0 & \text{otherwise.} \end{cases}$$

Here u_i is sampled from the uniform distribution in $[0, 1]$.

Check Constraint Satisfaction. To complete line 2 of Algorithm 1, given an assignment x^t at round $t \geq 1$, tensor W and matrix b , we compute Z^t as follows:

$$(2.12) \quad Z^t = W \circledast x^t + b,$$

where $\circledast$ represents a special multiplication between tensor and vector: $(W \circledast x)_{jk} = \sum_{i=1}^n W_{jki} x_i^t$. Note that $Z_{jk}^t = 1$ indicates the k -th literal of j -th clause is true (takes value 1). Hence, we compute S_j^t as:

$$(2.13) \quad S_j^t = 1 - \max_{1 \leq k \leq K} Z_{jk}^t, \quad \text{for } 1 \leq j \leq L.$$

Here $S_j^t = 1$ indicates x^t violates j -th clause. We check $\sum_{j=1}^L S_j^t \neq 0$ to see if any clause is violated, which corresponds to $C(x) = 0$ and is the continuation criteria of the while loop.

Extract Variables in Violated Clauses. To complete line 3 of Algorithm 1, we extract all the variables that require resampling based on vector S^t computed from the last step. The vector of resampling indicator A^t can be computed as:

$$(2.14) \quad A_i^t = \mathbf{1} \left(\sum_{j=1}^L S_j^t V_{ji} \geq 1 \right), \quad \text{for } 1 \leq i \leq n$$

where $\sum_{j=1}^L S_j^t V_{ji} \geq 1$ implies X_i requires resampling.

Resample. To complete line 4 of Algorithm 1, given the marginal probability vector P , resample indicator vector A^t and assignment x^t , we draw a new random sample x^{t+1} . This can be done using this update rule: for $1 \leq i \leq n$,

$$(2.15) \quad x_i^{t+1} = \begin{cases} (1 - A_i^t)x_i^t + A_i^t & \text{if } u_i > P_i, \\ (1 - A_i^t)x_i^t & \text{otherwise.} \end{cases}$$

Again, u_i is drawn from the uniform distribution in $[0, 1]$. Drawing multiple assignments in parallel is attained by extending x^t with a new dimension (See implementation in section 2.10.1). Example 1 show the detailed steps of NELSON (See more examples in section 2.7.5).

Example 1. Assume we have random variables X_1, X_2, X_3 with $n = 3$, Constraints $\mathcal{C} = (X_1 \vee X_2) \wedge (\neg X_1 \vee X_3)$ in the CNF form with $L = 2, K = 2$. Tensor W is:

$$W = \begin{bmatrix} w_{11}=[w_{111}, w_{112}, w_{113}], & w_{12}=[w_{121}, w_{122}, w_{123}] \\ w_{21}=[w_{211}, w_{212}, w_{213}], & w_{22}=[w_{221}, w_{222}, w_{223}] \end{bmatrix},$$

$$w_{11} = [1, 0, 0], w_{12} = [0, 1, 0], w_{21} = [-1, 0, 0], w_{22} = [0, 0, 1].$$

Algorithm 2 Learn Constrained MRFs via NELSON-CD.

Input: Dataset $\mathcal{D}$; Constraints $\mathcal{C}$; #Samples m ; Learning Iterations $T_{\max}$; Parameters of Constrained MRFs θ .

- 1: $\text{NELSON}(W, b, V) \leftarrow \text{build}(X, \mathcal{C})$. $\triangleright$ in Sec. 2.4
 - 2: **for** $t = 1$ **to** $T_{\max}$ **do**
 - 3: $\{x^j\}_{j=1}^m \sim \mathcal{D}$. $\triangleright$ from data
 - 4: $\{\tilde{x}^j\}_{j=1}^m \leftarrow \text{NELSON}(\theta^t, m)$. $\triangleright$ from model
 - 5: $g^t \leftarrow \frac{1}{m} \sum_{j=1}^m \nabla \phi(x^j) - \nabla \phi(\tilde{x}^j)$ $\triangleright$ divergence
 - 6: $\theta^{t+1} \leftarrow \theta^t - \eta g^t$. $\triangleright$ update parameters
 - 7: **end for** **return** The converged MRF model $\theta^{T_{\max}}$.
-

Note that $w_{111} = 1$ means X_1 is the 1st literal in the 1st clause and $w_{211} = -1$ means $\neg X_1$ is the 1st literal in the 2nd clause. Matrix b and the mapping matrix V are:

$$b = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix},$$

$b_{21} = 1$ indicates the 1st literal in the 2nd clause is negated. For the mapping matrix, $V_{11} = V_{12} = 1$ implies the 1st clause contains X_1 and X_2 . For $t = 1$, suppose we have an initialized assignment $x^1 = [0 \ 0 \ 1]^\top$, meaning $X_1 = X_2 = 0, X_3 = 1$. The intermediate results of Z^1, S^1, A^1 become:

$$Z^1 = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}, \quad S^1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad A^1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix},$$

where $S_1^1 = 1$ implies the 1st clause is violated. $A_1^1 = A_2^1 = 1$ denotes variables X_1, X_2 require resampling.

2.4.2. Contrastive Divergence-based Learning

The whole learning procedure is shown in Algorithm 2. At every learning iteration, we call NELSON to draw assignments $\{\tilde{x}^j\}_{j=1}^m$ from constrained MRF's distribution $P_\theta(X|\mathcal{C})$. Then we pick m data points at random from the training set $\{x^j\}_{j=1}^m \sim \mathcal{D}$. The divergence g^t in line 5 of Algorithm 2 is an estimation of $\nabla \ell_{\mathcal{C}}(\theta)$ in Eq. (2.4). Afterward, the MRFs' parameters are updated, according to line 6 of Algorithm 2. After $T_{\max}$ learning iterations, the algorithm outputs the constrained MRF model with parameters $\theta^{T_{\max}}$.

2.5. Experiments

We show the efficiency of the proposed NELSON on learning MRFs defined on the solutions of three combinatorial problems. Over all the tasks, we demonstrate that NELSON outperforms baselines on learning performance, i.e., generating structures with high likelihoods and MAP@10 scores (Table 2.1(c), 2.2(a,b), 2.3(c,d)). NELSON also generates samples which 100% satisfy constraints (Tables 2.1(b), 2.3(b)). Finally, NELSON is the most efficient sampler. Baselines either time out or cannot generate valid structures (Tables 2.1(a), 2.3(a)).

2.5.1. Experimental Settings

Baselines. We compare NELSON with other contrastive divergence learning algorithms equipped with other sampling approaches. In terms of baseline samplers, we consider:

- Gibbs sampler [Carter and Kohn, 1994], which is a special case of MCMC that is widely used in training MRF models.

- Weighted SAT samplers, including WAPS [Gupta et al., 2019], WeightGen [Chakraborty et al., 2014] and XOR sampler [Ermon et al., 2013; Ding and Xue, 2021].
- Uniform SAT samplers, including CMSGen [Golia et al., 2021], QuickSampler [Dutra et al., 2018], UniGen [Soos et al., 2020] and KUS [Sharma et al., 2018]. Notice these samplers cannot sample SAT solutions from a non-uniform distribution. We include them in the learning experiments as a comparison, and exclude them in the weighted sampling experiment (in Fig. 2.2).

Metrics. In terms of evaluation metrics, we consider:

- Training time per iteration, which computes the average time for every learning method to finish one iteration.
- Validness, that is the percentage of generated solutions that satisfy the given constraints $\mathcal{C}$.
- Mean Averaged Precision (MAP@10), which is the percentage that the solutions in the training set $\mathcal{D}$ reside among the top-10 *w.r.t.* likelihood score. The higher the MAP@10 scores, the better the model generates structures closely resembling those in the training set.
- log-likelihood of the solutions in the training set $\mathcal{D}$ (in Eq. 2.3). The model that attains the highest log-likelihood learns the closest distribution to the training set.
- Approximation error of $\nabla \log Z_{\mathcal{C}}(\theta)$, which is the L_1 distance between the exact value $\nabla \log Z_{\mathcal{C}}(\theta)$ and the approximated value given by the sampler.

See section 2.10 for detailed settings of baselines and evaluation metrics, as well as the following task definition, dataset construction, and potential function definition.

Table 2.1. Sampling efficiency and accuracy for learning K -SAT solutions with preferences. The proposed NELSON is the most efficient (see “Training Time Per Epoch”) and always generates valid assignments (see “Validness”) with a small approximation error (see “Approximation Error of Gradient”) against all baselines. T.O. means time out.

Problem size	(a) Training time per iteration (Mins) ($\downarrow$)								
	NELSON	XOR	WAPS	WeightGen	CMSGGen	KUS	QuickSampler	Unigen	Gibbs
10	0.13	26.30	1.75	0.64	0.22	0.72	0.40	0.66	0.86
20	0.15	134.50	3.04	T.O.	0.26	0.90	0.30	2.12	1.72
30	0.19	1102.95	6.62	T.O.	0.28	2.24	0.32	4.72	2.77
40	0.23	T.O.	33.70	T.O.	0.31	19.77	0.39	9.38	3.93
50	0.24	T.O.	909.18	T.O.	0.33	1532.22	0.37	13.29	5.27
500	5.99	T.O.	T.O.	T.O.	34.17	T.O.	T.O.	T.O.	221.83
1000	34.01	T.O.	T.O.	T.O.	177.39	T.O.	T.O.	T.O.	854.59
Problem size	(b) Validness of generated solutions (%) ($\uparrow$)								
	NELSON	XOR	WAPS	WeightGen	CMSGGen	KUS	QuickSampler	Unigen	Gibbs
10 – 50	100	100	100	100	100	100	82.65	100	90.58
500	100	T.O.	T.O.	T.O.	100	T.O.	7.42	100	54.27
1000	100	T.O.	T.O.	T.O.	100	T.O.	0.00	100	33.91
Problem size	(c) Approximation error of $\nabla \log Z_C(\theta)$ ($\downarrow$)								
	NELSON	XOR	WAPS	WeightGen	CMSGGen	KUS	QuickSampler	Unigen	Gibbs
10	0.10	0.21	0.12	3.58	3.96	4.08	3.93	4.16	0.69
12	0.14	0.19	0.16	5.58	5.50	5.49	5.55	5.48	0.75
14	0.15	0.25	0.19	T.O.	6.55	6.24	7.79	6.34	1.30
16	0.16	0.25	0.15	T.O.	9.08	9.05	9.35	9.03	1.67
18	0.18	0.30	0.23	T.O.	10.44	10.30	11.73	10.20	1.90

2.5.2. Random K -SAT Solutions with Preference

Task Definition & Dataset. This task is to learn to generate solutions to a K -SAT problem. We are given a training set $\mathcal{D}$ containing solutions to a corresponding CNF formula $c_1 \wedge \dots \wedge c_L$. Note that not all solutions are equally likely to be presented in $\mathcal{D}$. The *learning* task is to maximize the log-likelihood of the assignments seen in the training set $\mathcal{D}$. Once learning is completed, the *inference* task is to generate valid solutions that closely resemble those in $\mathcal{D}$ [Dodaro and Previti, 2019]. To generate the training set $\mathcal{D}$, we use CNFGen [Lauria et al., 2017] to generate the random K -SAT problem and use Glucose4 solver to generate random valid solutions [Ignatiev et al., 2018].

Sampler’s Efficiency and Accuracy. Table 2.1 shows the proposed NELSON is an efficient sampler that generates valid assignments, in terms of the training time for learning

Table 2.2. The quality of learning outcomes for learning random K -SAT solutions with preferences. NELSON achieves the best likelihood and MAP@10 scores. T.O. is time out.

	(a) log-likelihood ($\uparrow$)			
Problem size	NELSON	Gibbs	CMSGen	Quicksampler WeightGen,KUS XOR, WAPS
100	-49.16	- 36.36	-60.12	T.O.
300	- 52.61	-53.11	-128.39	
500	- 196.47	-197.21	-272.49	
700	- 238.60	-238.75	-389.44	
1000	- 294.22	-296.33	-532.85	
	(b) MAP@10 (%) ($\uparrow$)			
100	82.13	83.32	86.34	T.O.
300	66.37	64.42	64.50	
500	90.03	73.14	70.67	
700	69.74	69.74	48.10	
1000	91.70	77.56	78.72	

constrained MRF, approximation error for the gradient and validness of the generated assignments. In Table 2.1(a), NELSON takes much less time for sampling against all the samplers and can train the model with the dataset of problem size 1000 within an hour. In Table 2.1(b), NELSON always generates valid samples. The performance of QuickSampler and Gibbs methods decreases when the problem size becomes larger. In Table 2.1(c), NELSON, XOR and WAPS are the three algorithms that can effectively estimate the gradient while the other algorithms incur huge estimation errors. Also, the rest methods are much slower than NELSON.

Learning Quality Table 2.2 demonstrates NELSON-CD learns a more accurate constrained MRF model by measuring the log-likelihood and MAP@10 scores. Note that baselines including Quicksampler, Weightgen, KUS, XOR and WAPS timed out for the problem sizes we considered. Compared with the remaining baselines, NELSON attains the best log-likelihood and MAP@10 metric.

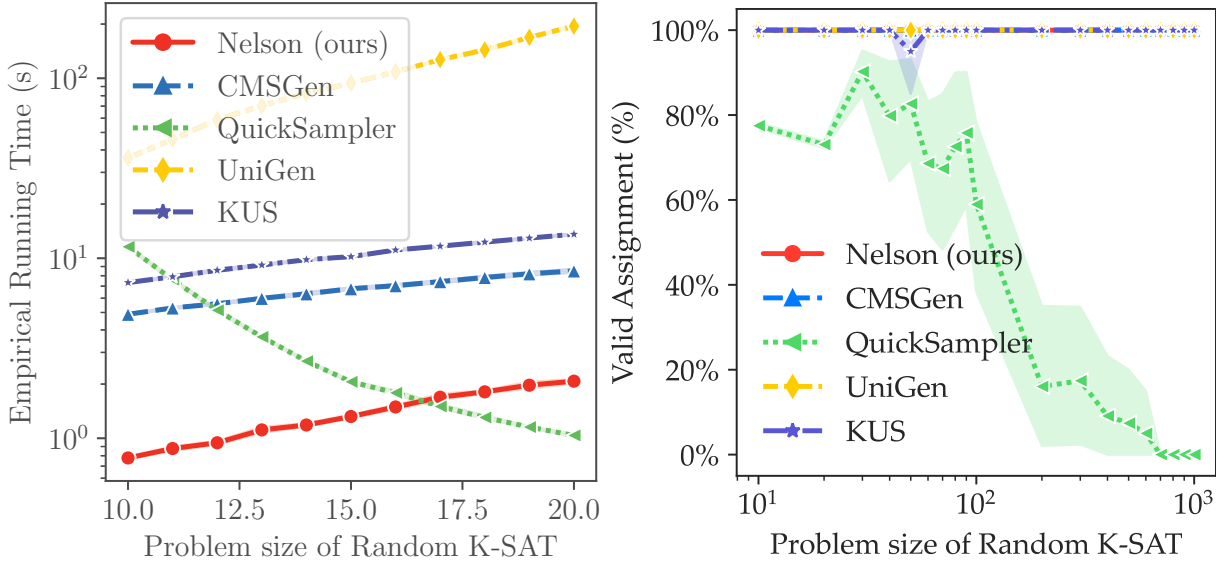


Figure 2.1. Running time and the percentage of valid structures sampled uniformly at random from solutions of K-SAT problems. Among all the problem sizes, NELSON always generate valid solutions and is the most efficient sampler.

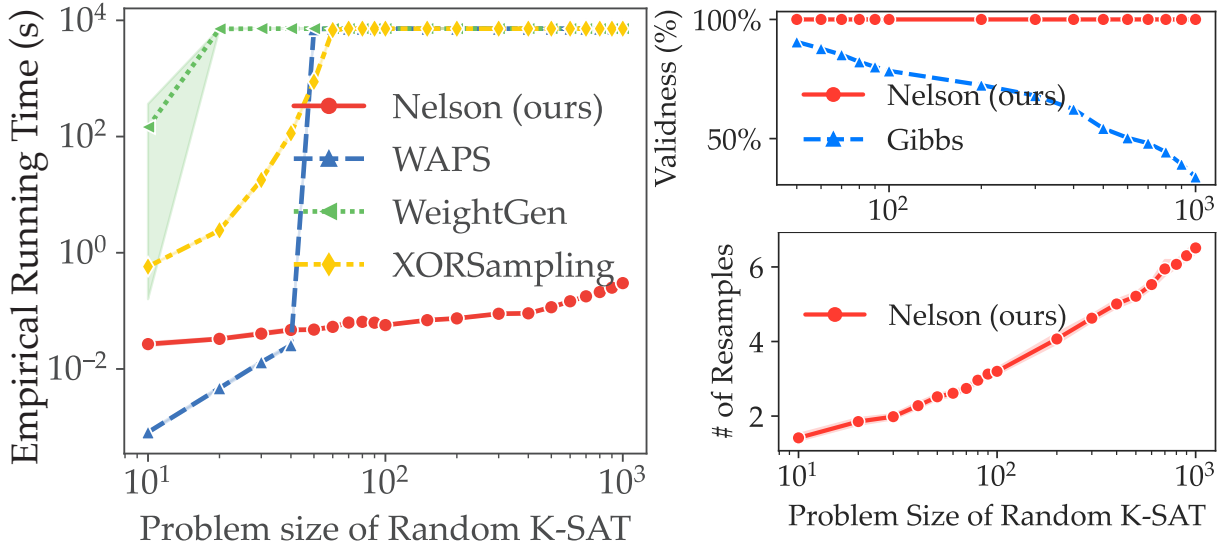


Figure 2.2. Running time, the percentage of valid solutions generated, and rounds of resampling for weighted sample generation of K-SAT solutions. Among all the problem sizes, NELSON scales the best among all approaches and always generates valid solutions.

Abalation Study. We also evaluated the samplers’ efficiency in isolation (not embedded in learning). The sampling cases we considered are uniform and weighted (mainly following the experiment setting in [Chakraborty and Meel, 2019]). In weighted sampling, the weights are specified by fixed values to the single factors in Eq. (2.6). In the uniform sampling case in Fig. 2.1, NELSON and Quicksampler require much less time to draw samples compared to other approaches. However, the solutions generated by Quicksampler rarely satisfy constraints. In the weighted sampling case in Fig. 2.2, NELSON scales better than all the competing samplers as the sizes of the K -SAT problems increase.

2.5.3. Sink-Free Orientation in Undirected Graphs

Task Definition & Dataset A *sink-free* orientation of an undirected graph is a choice of orientation for each arc such that every vertex has at least one outgoing arc [Cohn et al., 2002]. This task has wide applications in robotics routing and IoT network configuration [Takahashi et al., 2009]. Even though finding a sink-free orientation is tractable, sampling a sink-free orientation from the space of all orientations is still $\#P$ -hard. Given a training set of preferred orientations $\mathcal{D}$ for the graph, the *learning* task is to maximize the log-likelihood of the orientations seen in the training set. The *inference* task is to generate valid orientations that resemble those in the training set. To generate the training set, we use the Erdős-Rényi random graph from the NetworkX library. The problem size is characterized by the number of vertices in the graph. The baselines we consider are CD-based learning with Gibbs sampling and CMSGen.

Learning Quality. In Table 2.3(a), we show the proposed NELSON method takes much less time to train MRF for one epoch than the competing approaches. Furthermore, in

Table 2.3. Sample efficiency and learning performance of the sink-free orientation task. NELSON is the most efficient (see Training Time Per Epoch) and always generates valid assignments (see Validness), has the smallest error approximating gradients, and has the best learning performance (see MAP@10) among all baselines.

Problem size	(a) Training Time Per Epoch (Mins) ($\downarrow$)		
	NELSON	Gibbs	CMSGen
10	0.53	9.85	0.69
20	0.53	80.12	1.93
30	0.72	256.38	3.65
40	0.93	777.01	5.99
50	1.17	T.O.	9.08
	(b) Validness of Orientations (%) ($\uparrow$)		
7	100	50.16	100
8	100	64.63	100
9	100	47.20	100
10	100	62.60	100
11	100	84.95	100
	(c) Approximation Error of $\nabla \log Z_C(\theta)$ ($\downarrow$)		
5	0.01	0.09	0.21
7	0.05	0.08	2.37
9	0.03	0.11	2.37
11	0.04	0.17	8.62
13	0.05	0.28	11.27
	(d) MAP@10 (%) ($\uparrow$)		
10	61.14	60.01	64.56
20	55.26	55.20	47.79
30	100.00	96.29	100.00
40	40.01	39.88	38.90
50	46.12	T.O.	42.11

Table 2.3(b), NELSON and CMSGen generate 100% valid orientations of the graph while the Gibbs-based model does not. Note the constraints for this task satisfy Condition 1, hence NELSON sampler’s performance is guaranteed by Theorem 1. In Table 2.3(c), NELSON attains the smallest approximation error for the gradient (in Eq. 2.4) compared to baselines. Finally, NELSON learns a higher MAP@10 than CMSGen. The Gibbs-based

approach times out for problem sizes larger than 40. In summary, our NELSON is the best-performing algorithm for this task.

2.5.4. Learn Vehicle Delivery Routes

Task Definition & Dataset. Given a set of locations to visit, the task is to generate a sequence to visit these locations in which each location is visited once and only once and the sequence closely resembles the trend presented in the training data. The training data are such routes collected in the past. The dataset is constructed from TSPLIB, which consists of 29 cities in Bavaria, Germany. The constraints for this problem do not satisfy Condition 1. We still apply the proposed method to evaluate if the NELSON algorithm can handle those general hard constraints.

In Fig. 2.3, we see NELSON can obtain samples of this delivery problem efficiently. We measure the number of resamples taken as well as the corresponding time used by the NELSON method. NELSON takes roughly 50 times of resamples with an average time of 0.3 seconds to draw a batch (the batch size is 100) of valid visiting sequences.

2.6. Chapter Recap

Before we move onto the more technical part, let us recap the research problems this chapter wants to address. In this chapter, we present NELSON, which embeds a sampler based on Lovász Local Lemma into the contrastive divergence learning of Markov random fields. The embedding is fully differentiable. This approach allows us to learn generative models over constrained domains, which presents significant challenges to other state-of-the-art models. We also give sound proofs of the performance of the LLL-based sampler.

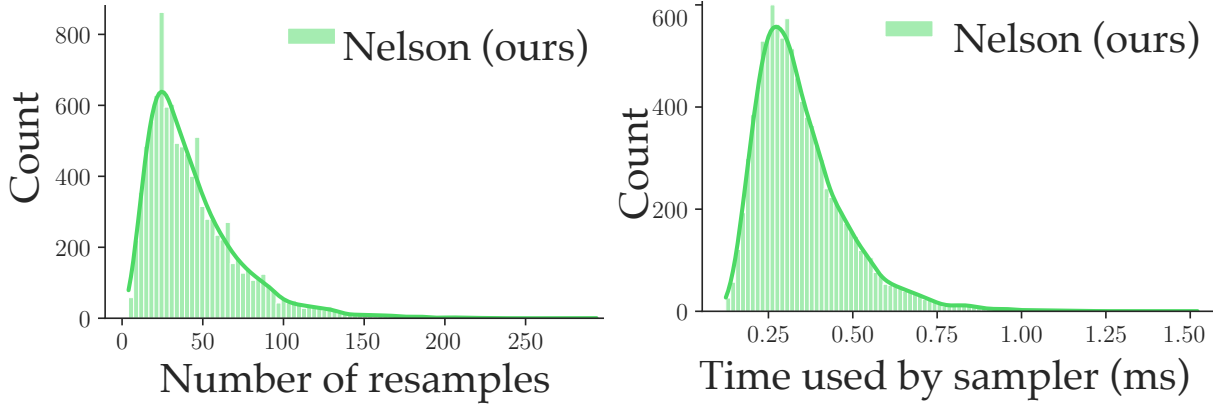


Figure 2.3. Frequency histograms for the number of resample and the total time of NELSON method for uniformly sampling visiting paths for vehicle routing problem.

Experimental results on several real-world domains reveal that NELSON learns to generate 100% valid structures, while baselines either time out or cannot generate valid structures. NELSON also outperforms other approaches in the running times and in various learning metrics.

2.7. Probability Distribution of Algorithm 1

2.7.1. Definitions and Notations

This section is for the proofs related to the probability distribution in the proposed Algorithm 1. For convenience, commonly used notations are listed in Table 2.4. We make some slight changes to some notations that appear in this chapter to make sure they are consistent and well-defined in this proof.

Similar to the previous analysis [Guo et al., 2019; Jerrum, 2021], we begin by introducing the concept “dependency graph” (in Definition 1) for the constraints $\mathcal{C}$.

Definition 1 (Dependency Graph). *The dependency graph $G = (\mathcal{C}, E)$, where the vertex set is the set of constraints $\mathcal{C}$. Two vertices c_i and c_j are connected with an edge $(c_i, c_j) \in E$ if and only if they are defined on at least one common random variable, i.e., $\text{var}(c_i) \cap \text{var}(c_j) \neq \emptyset$.*

For keeping track of the whole sampling procedure, we need the concept “sampling record” (in Definition 2) [Guo et al., 2019], which are the broken constraints at every round in Algorithm 1. It is also known as the witness tree in [Moser and Tardos, 2010]. This allows us to check the constraint satisfaction of the assignment at every round.

Under Condition 1, for any edge in the dependency graph $(c_i, c_j) \in E$, either $\mathbf{1}(x, c_i) = 0$ or $\mathbf{1}(x, c_j) = 0$ for all $x \in \mathcal{X}$. In other words, two constraints with shared related variables, representing two *adjacent vertices* in the dependency graph G , are not broken simultaneously. Thus, the constraints in record S_t form an *independent set*² over the dependency graph under Condition 1.

Definition 2 (Sampling Record). *Given dependency graph $G(\mathcal{C}, E)$, let $X_t = x$ be one possible assignment obtained at round t of Algorithm 1. Let $S_t \subseteq \mathcal{C}$ be the set of vertices in graph G (subset of constraints) that x violates*

$$(2.16) \quad S_t = \{c_i | c_i \in \mathcal{C} \text{ and } \mathbf{1}(x, c_i) = 0\},$$

where indicator function $\mathbf{1}(x, c_i) = 0$ implies x violates constraint c_i at round t . Define the sampling record as the sequence of violated constraints $S_1, \dots, S_t$ throughout the execution.

²A set of vertices with no two adjacent vertices in the graph.

At round t ($t \geq 1$) of Algorithm 1, suppose the violated constraints is $S_t \subseteq \mathcal{C}$. The constraints that are not adjacent to S_t in the dependency graph are still satisfied after re-sample. The only possible constraints that might be broken after the re-sample operation are among S_t itself, or those constraints directly connected to S_t in the dependency graph. Therefore,

$$S_{t+1} \subset \Gamma(S_t), \quad \text{for all } t \geq 1.$$

where $\Gamma(S_t)$ is the set of vertices of S_t and its adjacent neighbors in the dependency graph G (see Table 2.4). When Algorithm 1 terminates at round $T + 1$, no constraints are violated anymore, i.e., $s_{T+1} = \emptyset$. To summarize the above discussion on a sampling record by Algorithm 1, we have the following Claim 1.

Claim 1. *Under Condition 1, a potential sampling record of length $T + 1$ by the Algorithm 1 is a sequence of independent sets: $S_1, S_2, \dots, S_T, \emptyset$ with*

- (1) $S_{t+1} \subseteq \Gamma(S_t)$ and $S_t \neq \emptyset$, for $1 \leq t \leq T$;
- (2) $s_{T+1} = \emptyset$.

Extra Notations related to Constrained MRF. The constrained MRF model over constraints set $\mathcal{C}$ is defined as:

$$P_\theta(X = x|\mathcal{C}) = \frac{\exp(\sum_{i=1}^n \theta_i x_i) \mathbf{1}(x, \mathcal{C})}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C})}$$

where the partition function only sums over valid assignments in $\mathcal{X}$. Note that $C(x)$ in Equation (2.6) is the same as $\mathbf{1}(x', \mathcal{C})$ in the above equation. We slightly change the notations for consistency in this proof. Also notice that the output distribution can no

Table 2.4. Summary of all the notations used in the theoretical analysis of Algorithm 1.

Notation	Definition
$X = \{X_i\}_{i=1}^n$	set of discrete random variables
$x \in \mathcal{X}$	possible assignments for variables X
$x_i \in \mathcal{X}_i$	variable X_i can take all values in $\mathcal{X}_i$
$\mathcal{C} = \{c_j\}_{j=1}^m$	given constraints
$S_t \subseteq \mathcal{C}$	subset of constraints violated at round t of Algorithm 1
$G(\mathcal{C}, E)$	the dependency graph (in Definition 1)
$\text{var}(c_j)$	the indices of domain variables that are related to constraint c_i
$\text{var}(S_t)$	the indices for domain variables that are related to constraints S_t
$\Gamma(c_j)$	c_j and its direct neighbors in the dependency graph
$\Gamma(S_t)$	S_t and direct neighbors of S_t in the dependency graph
$\mathcal{C} \setminus \Gamma(S_t)$	all constraints in $\mathcal{C}$ but not in $\Gamma(S_t)$
$S_1, \dots, S_T, \emptyset$	a sampling record of Algorithm 1 (in Definition 2)
$\mathbf{1}(x, c_i)$	indicator function that evaluates if assignment x satisfies constraint c_i
$\mathbf{1}(x, S_t)$	indicator function that evaluates if assignment x satisfies constraints in S_t
$\mathbf{1}(x, \mathcal{C})$	indicator function that evaluates if assignment x satisfies all constraints $\mathcal{C}$
$P_\theta(X \mathcal{C} \setminus \Gamma(S_t))$	see Definition 3
$\mathbb{P}(X S_t)$	see Definition 4

longer be factorized after constraints are enforced, since the partition function cannot be factorized. Our task is to draw samples from this distribution.

To analyze the intermediate steps in Algorithm 1, we further need to define the following notations.

Definition 3. *The constrained MRF distribution for constraints $\mathcal{C} \setminus \Gamma(S_t)$ is*

$$P_\theta(X = x|\mathcal{C} \setminus \Gamma(S_t)) = \frac{\exp(\sum_{i=1}^n \theta_i x_i) \mathbf{1}(x, \mathcal{C} \setminus \Gamma(S_t))}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C} \setminus \Gamma(S_t))}$$

Definition 4. *At round t of Algorithm 1, assume $S_t \subseteq \mathcal{C}$ are the set of broken constraints, Define $\mathbb{P}(X_{t+1} = x|S_1, \dots, S_t)$ to be the probability of obtaining a new assignment x after we re-sample random variables indexed by $\text{var}(S_t)$.*

2.7.2. Ratio Property Lemma

Lemma 1 (Ratio Property). *Under Condition 1, assume Algorithm 1 is at round t . Conditioning on observing one possible sampling record $S_1, \dots, S_t$, Algorithm 1 step 4 will re-sample variables in $\text{var}(S_t)$ at round $t + 1$. Let $x, x' \in \mathcal{X}$ be two possible assignments after this re-sample. The probability ratio of obtaining these two results equals that under constrained MRF $P_\theta(x|\mathcal{C} \setminus \Gamma(S_t))$:*

$$(2.17) \quad \frac{\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)}{\mathbb{P}(X_{t+1} = x' | S_1, \dots, S_t)} = \frac{P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t))}{P_\theta(X = x' | \mathcal{C} \setminus \Gamma(S_t))},$$

where $\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)$ is the probability of Algorithm 1 step 4 produces assignment x at round $t + 1$, conditioning on the observed record $S_1, \dots, S_t$ and re-sample $\text{var}(S_t)$. $P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t))$ is the constrained MRF (for the constraints $\mathcal{C} \setminus \Gamma(S_t)$) probability on assignment x .

PROOF. During the intermediate step of the algorithm, assume the set of constraints S_t are violated. We want to re-sample variables indexed by $\text{var}(S_t)$, so variables indexed by $\text{var}(\mathcal{C} \setminus \Gamma(S_t))$ won't change assignments. Also, because $\Gamma(S_t)$ is the largest possible set of constraints that can be infected by the re-sample, constraints $\mathcal{C} \setminus \Gamma(S_t)$ are still satisfied after the re-sample.

At round t , we re-sample variables in $\text{var}(S_t)$ according to step 4 in Algorithm 1, we thus have:

$$\mathbb{P}(X_{\text{var}(S_t)}^{t+1} = x_{\text{var}(S_t)} | S_1 \dots S_t) = \prod_{i \in \text{var}(S_t)} \frac{\exp(\theta_i x_i)}{\sum_{x'_i \in \mathcal{X}_i} \exp(\theta_i x'_i)}.$$

Here the notation $X_{\text{var}(S_t)}^{t+1} = x_{\text{var}(S_t)}$ means $X_i = x_i$ for $i \in \text{var}(S_t)$ at round t . For any two possible assignments x, x' after the re-sample,

$$x_i = x'_i, \quad \text{for } i \in \{1, \dots, n\} \setminus \text{var}(S_t)$$

since the rest variable's assignments are kept the same after re-sample. Thus, we can have ratio:

$$(2.18) \quad \frac{\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)}{\mathbb{P}(X_{t+1} = x' | S_1, \dots, S_t)} = \frac{\exp(\sum_{i \in \text{var}(S_t)} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(S_t)} \theta_i x'_i)} = \frac{\exp(\sum_{i \in \text{var}(\Gamma(S_t))} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(\Gamma(S_t))} \theta_i x'_i)}.$$

For the last step, since every assignment outside $\text{var}(S_t)$ is not changed, we can enlarge the index set of summation to $\Gamma(S_t)$ by multiplying

$$1 = \frac{\exp(\sum_{i \in \text{var}(\Gamma(S_t)) \setminus \text{var}(S_t)} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(\Gamma(S_t)) \setminus \text{var}(S_t)} \theta_i x'_i)}.$$

After re-sample, we know x must satisfy the constraints $\mathcal{C} \setminus \Gamma(S_t)$. Thus, the probability of this x conditioned on constraints $\mathcal{C} \setminus \Gamma(S_t)$ holding in the constrained MRF model is:

$$\begin{aligned} P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t)) &= \frac{\exp(\sum_{i=1}^n \theta_i x_i) \mathbf{1}(x, \mathcal{C} \setminus \Gamma(S_t))}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C} \setminus \Gamma(S_t))} \\ &= \frac{\exp(\sum_{i=1}^n \theta_i x_i)}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C} \setminus \Gamma(S_t))}. \end{aligned}$$

In the constrained MRF model (for constraints $\mathcal{C} \setminus \Gamma(S_t)$), the ratio of these two probabilistic assignments x, x' is:

$$(2.19) \quad \frac{P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t))}{P_\theta(X = x' | \mathcal{C} \setminus \Gamma(S_t))} = \frac{\exp(\sum_{i \in \text{var}(\Gamma(S_t))} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(\Gamma(S_t))} \theta_i x'_i)},$$

because the x_i outside $\text{var}(\Gamma(S_t))$ remains the same. Note that x, x' are two possible assignments produced according to step 4 in Algorithm 1 at round t . Combining Equation (2.18) and Equation (2.19), we conclude that:

$$\frac{\mathbb{P}(X_{t+1} = x | S_1, \dots, S_t)}{\mathbb{P}(X_{t+1} = x' | S_1, \dots, S_t)} = \frac{P_\theta(X = x | \mathcal{C} \setminus \Gamma(S_t))}{P_\theta(X = x' | \mathcal{C} \setminus \Gamma(S_t))}.$$

The proof is finished. $\square$

2.7.3. Proof of Theorem 1

Suppose the re-sampling process terminates at round $T + 1$ and we obtain a valid sample x . Upon the termination of Algorithm 1, all the constraints are satisfied. So we have: $S_{T+1} = \emptyset$. In other words, $\mathbf{1}(x, \mathcal{C}) = 1$.

Let x, x' be two possible valid assignments produced at round $T + 1$ by the Algorithm 1. Using the analysis in Lemma 1, we can still have:

$$\frac{\mathbb{P}(X_{T+1} = x | S_1, \dots, S_T)}{\mathbb{P}(X_{T+1} = x' | S_1, \dots, S_T)} = \frac{\exp(\sum_{i \in \text{var}(S_T)} \theta_i x_i)}{\exp(\sum_{i \in \text{var}(S_T)} \theta_i x'_i)}.$$

The probability of this x in the constrained MRF model (for constraints $\mathcal{C}$) is:

$$P_\theta(X = x | \mathcal{C}) = \frac{\exp(\sum_{i=1}^n \theta_i x_i) \mathbf{1}(x, \mathcal{C})}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C})} = \frac{\exp(\sum_{i=1}^n \theta_i x_i)}{\sum_{x' \in \mathcal{X}} \exp(\sum_{i=1}^n \theta_i x'_i) \mathbf{1}(x', \mathcal{C})}.$$

Then we conclude that:

$$\frac{\mathbb{P}(X_{T+1} = x | S_1, \dots, S_T)}{\mathbb{P}(X_{T+1} = x' | S_1, \dots, S_T)} = \frac{P_\theta(X = x | \mathcal{C})}{P_\theta(X = x' | \mathcal{C})}.$$

Note that this ratio property holds for all the possible sampling records $S_1, \dots, S_T, \emptyset$.

Summation of All Possible Sampling Records. Define $\mathbb{P}(S_1, \dots, S_T)$ to be the probability of observing record $S_1, \dots, S_T$ by Algorithm 1. For any possible sampling record $S_1, \dots, S_T, \emptyset$, the ratio property still holds:

$$\frac{\mathbb{P}(X_{T+1} = x | S_1, \dots, S_T) \mathbb{P}(S_1, \dots, S_T)}{\mathbb{P}(X_{T+1} = x' | S_1, \dots, S_T) \mathbb{P}(S_1, \dots, S_T)} = \frac{P_\theta(X = x | \mathcal{C})}{P_\theta(X = x' | \mathcal{C})}$$

where the term $\mathbb{P}(S_1, \dots, S_T)$ on the Left-hand-side (LHS) is actually the same. After we summarize over all possible sampling records $S_1, \dots, S_T, \emptyset$, the ratio property still holds. Let $\mathbb{P}(X_{T+1} = x)$ be the probability of obtaining one valid assignment x by the execution of Algorithm 1.

(2.20)

$$\frac{\mathbb{P}(X_{T+1} = x)}{\mathbb{P}(X_{T+1} = x')} = \frac{\sum_{S_1, \dots, S_T} \mathbb{P}(X_{T+1} = x | S_1, \dots, S_T) \mathbb{P}(S_1, \dots, S_T)}{\sum_{S_1, \dots, S_T} \mathbb{P}(X_{T+1} = x' | S_1, \dots, S_T) \mathbb{P}(S_1, \dots, S_T)} = \frac{P_\theta(X = x | \mathcal{C})}{P_\theta(X = x' | \mathcal{C})}$$

Sample Space Analysis At Termination. We need one more statement to show Theorem 1 holds. Let $\mathcal{X}_{\text{LLL}}$ be the set of all possible assignments x that can be generated by Algorithm 1:

$$\mathcal{X}_{\text{LLL}} = \bigcup_{S_1 \dots S_T} \{x | \mathbb{P}(X_{T+1} = x | S_1 \dots S_T) \neq 0 \text{ and } \mathbb{P}(S_1 \dots S_T) \neq 0\}.$$

where $\mathbb{P}(S_1 \dots S_T) \neq 0$ means $S_1, \dots, S_T$ is a possible record. $\mathbb{P}(X_{T+1} = x | S_1 \dots S_T) \neq 0$ means it is possible to obtain x given the record $S_1, \dots, S_T$.

Let $\mathcal{X}_{\mathcal{C}}$ be the set of assignments x that satisfy all the constraints in the constrained MRF (for constraints $\mathcal{C}$):

$$\mathcal{X}_{\mathcal{C}} = \{x | P_\theta(X = x | \mathcal{C}) \neq 0, \text{ for all } x \in \mathcal{X}\}.$$

Lemma 2. $\mathcal{X}_{LLL} \subseteq \mathcal{X}_C$ and $\mathcal{X}_C \subseteq \mathcal{X}_{LLL}$, thus $\mathcal{X}_{LLL} = \mathcal{X}_C$.

PROOF. When Algorithm 1 terminates, it only produces valid assignments; thus, we must have: $\mathcal{X}_{LLL} \subseteq \mathcal{X}_C$. On the other hand, there is always a non-zero probability that Algorithm 1 will generate every valid assignment $x \in \mathcal{X}_C$, which implies that $\mathcal{X}_C \subseteq \mathcal{X}_{LLL}$. Therefore we can conclude that $\mathcal{X}_{LLL} = \mathcal{X}_C$. $\square$

Lemma 2 show that the two distributions have the same sample space when Algorithm 1 terminates. What's more, Equation (2.20) shows they have the same probability ratio for any possible valid assignments x, x' . This shows that the execution of the Algorithm 1 is a random draw from the constrained MRF distribution $P_\theta(X = x|\mathcal{C})$. The proof of Theorem 1 is finished.

2.7.4. Difference to the Original Proof

The main difference in the above proof to the existing proof in [Guo et al., 2019, Lemma 7] is that: We show Lemma 1 that characterizes the proportional ratio of getting different assignments of variables, which is more general than the descriptive proof for [Guo et al., 2019, Lemma 7].

2.7.5. A Running Example in View of Markov Chain

We dedicate this section to demonstrate the execution of Algorithm 1 with Example 1. Algorithm 1 can be viewed as a Markov chain, so we will show the probability of obtaining valid samples is unbiased by running thousands of steps of the Markov chain. The constraints are $\mathcal{C} = \{c_1 = (X_1 \vee X_2), c_2 = (\neg X_1 \vee X_3)\}$. We use the assignment of all the

variables as the states $s_1, \dots, s_8$ in the rounds of Algorithm 1.

$$\begin{aligned}
 s_1 &= (X_0 = 0, X_1 = 0, X_2 = 0) \\
 s_2 &= (X_0 = 0, X_1 = 0, X_2 = 1) \\
 s_3 &= (X_0 = 0, X_1 = 1, X_2 = 0) \\
 s_4 &= (X_0 = 0, X_1 = 1, X_2 = 1) \\
 s_5 &= (X_0 = 1, X_1 = 0, X_2 = 0) \\
 s_6 &= (X_0 = 1, X_1 = 0, X_2 = 1) \\
 s_7 &= (X_0 = 1, X_1 = 1, X_2 = 0) \\
 s_8 &= (X_0 = 1, X_1 = 1, X_2 = 1)
 \end{aligned}
 \tag{2.21}$$

Here s_1, s_2, s_3, s_4 correspond to *valid* assignments of variables with respect to the constraints $\mathcal{C}$ and s_5, s_6, s_7, s_8 correspond to *invalid* assignments of variables, that requires resampling.

For simplicity, we consider the uniform setting where $\theta_1 = \theta_2 = \theta_3$. The goal is to sample every valid assignment with equal probability. Therefore, the probability for every variable is:

$$P(X_i) = \begin{cases} \frac{1}{2} & \text{for variable } X_i \text{ taking value 1} \\ \frac{1}{2} & \text{for variable } X_i \text{ taking value 0} \end{cases}$$

for $i = 1, 2, 3$. Based on Algorithm 1, we know the probability of transferring from s_i to s_j ($1 \leq i, j \leq 8$). Thus we can construct the transition matrix between every state:

$$(2.22) \quad T = \begin{bmatrix} & s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 \\ s_1 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ s_2 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ s_3 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\ s_4 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\ s_5 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} & 0 & 0 & 0 \\ s_6 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} \\ s_7 & \frac{1}{4} & 0 & \frac{1}{4} & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 \\ s_8 & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{bmatrix}$$

where $T_{ij} = T(s_i, s_j)$ denotes the transition probability from state s_i to state s_j .

Taking state s_5 as an example, it violates constraint C_2 thus X_2, X_3 will be resampled. There are 4 possible assignments of X_2, X_3 , which corresponds to states $\{s_1, s_2, s_3, s_5\}$. Since each variable is resampled uniformly at random, the probability of transition from state s_5 to the states $\{s_1, s_2, s_3, s_5\}$ are $1/4$. The Algorithm 1 will terminate once it reaches states $\{s_1, s_2, s_3, s_4\}$, which corresponds to the (valid) state only transit to itself with probability 1. Thus we find $T(s_i, s_i) = 1$ for $i = 1, 2, 3, 4$.

For a randomly initialized assignment:

[illegible]

that has an equal probability of being any state. After executing Algorithm 1 for 2000 steps, we have:

$$(2.24) \quad T^{2000} = \begin{bmatrix} & s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 \\ s_1 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ s_2 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ s_3 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\ s_4 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\ s_5 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 & 0 & 0 \\ s_6 & \frac{1}{6} & \frac{1}{2} & \frac{1}{6} & \frac{1}{6} & 0 & 0 & 0 & \frac{1}{4} \\ s_7 & \frac{1}{3} & 0 & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 & 0 \\ s_8 & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{2} & 0 & 0 & 0 & 0 \end{bmatrix},$$

$$(2.25) \quad xT^{2000} = \begin{bmatrix} s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & 0 & 0 & 0 \end{bmatrix}.$$

This implies Algorithm 1 outputs every valid assignment with the same probability in the uniform setting, which follows the result in Theorem 1.

2.8. Running Time Analysis of Algorithm 1

We dedicate this section to showing the running time of Algorithm 1 on a general weighted case. The expected running time of Algorithm 1 is determined by the number of rounds of re-sampling. Algorithm 1 re-sample all the related random variables simultaneously in every single round. However, it is hard to get an estimation of the exact

total running time over the *random variables*. Instead, we can only have a loose upper bound of the expected running time over the *sequence of sampling record* (the sequence of violated constraints).

The overall structure of the proof is similar to the proof in [Guo et al., 2019, Theorem 13]. We show the difference in our proof at the end of this section.

2.8.1. Definitions and Notations

We define the following terms to simplify our notations.

Definition 5. Let S_t be a subset of vertices in a dependency graph. 1) Define p_{S_t} as the probability of constraints in S_t being violated:

$$(2.26) \quad p_{S_t} = \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \right)$$

where we use $\neg c_i$ to indicate the constraint c_i is violated. 2) Define q_{S_t} as the probability that only the constraints in S_t are violated and nothing else.

$$(2.27) \quad q_{S_t} = \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \right)$$

where $\bigwedge_{c_i \in S_t} \neg c_i$ corresponds to only the constraints in S_t are violated and $\bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j$ corresponds to all the rest constraints are satisfied. So $q_{\{c_i\}}$ is the probability that only constraint c_i is broken and all the rest still hold. Similarly, $q_{\emptyset}$ denotes the probability that all the constraints are satisfied.

Lemma 3. *Given Definition 5, we can further expand q_{S_t} under Condition 1:*

$$q_{S_t} = p_{S_t} \mathbb{P} \left(\bigwedge_{c_j \in \mathcal{C} \setminus \Gamma(S_t)} c_j \right)$$

PROOF. We can split q_{S_t} into the probability of two independent events:

$$\begin{aligned} q_{S_t} &= \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \right) && \text{By definition of } q_{S_t} \text{ in Equation (2.27)} \\ &= \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus \Gamma(S_t)} c_j \right) \\ &= \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \right) \mathbb{P} \left(\bigwedge_{c_n \in \mathcal{C} \setminus \Gamma(S_t)} c_n \right) \\ &= p_{S_t} \mathbb{P} \left(\bigwedge_{c_j \in \mathcal{C} \setminus \Gamma(S_t)} c_j \right). && \text{By definition of } p_{S_t} \text{ in Equation (2.26)} \end{aligned}$$

The second equality holds because under Condition 1, adjacent vertices have zero probability. In other words, when we observe that constraints in S_t are violated, constraints in $\Gamma(S_t) \setminus S_t$ cannot be violated. The third equality holds because the random variables in $\mathbf{var}(S_t)$ are *independent to* those variables in $\mathbf{var}(\mathcal{C} \setminus \Gamma(S_t))$. So we can apply $P(AB) = P(A)P(B)$ when the events A, B are independent to each other. $\square$

Remark 1 (Equivalence of Record). *At round t of Algorithm 1, it finds all the constraints S_t that are broken ($\bigwedge_{c_i \in S_t} \neg c_i$), which implies the rest of the constraints $\mathcal{C} \setminus \Gamma(S_t)$ are satisfied ($\bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j$). Thus the probability of observing S_t in the record is equivalent*

to the following:

$$(2.28) \quad \mathbb{P}(S_t) = \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \right)$$

Lemma 4. *Given a possible sampling record $S_1 \dots S_{t-1}, S_t$ by Algorithm 1, the following equality holds for the pair (S_{t-1}, S_t) :*

$$\sum_{S_t} q_{S_t} = \mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)$$

PROOF. By Definition 2 of the sampling record, we have $S_t \subset \Gamma(S_{t-1})$. The relationship of its complement would be:

$$\mathcal{C} \setminus \Gamma(S_{t-1}) \subset \mathcal{C} \setminus S_t.$$

Using the above result, we have:

$$(2.29) \quad \mathbb{P} \left(\bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \wedge \bigwedge_{c_k \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_k \right) = \mathbb{P} \left(\bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \right)$$

Based on Remark 1 and Baye's theorem, we have:

$$\begin{aligned}
& \mathbb{P}(S_t \mid \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i) \\
&= \mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \mid \bigwedge_{c_k \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_k \right) && \text{By Equation (2.28)} \\
&= \frac{\mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_j \in \mathcal{C} \setminus S_t} c_j \wedge \bigwedge_{c_k \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_k \right)}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)} && \text{By Bayes's formula} \\
&= \frac{\mathbb{P} \left(\bigwedge_{c_i \in S_t} \neg c_i \wedge \bigwedge_{c_k \in \mathcal{C} \setminus S_t} c_k \right)}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)} && \text{By Equation (2.29)} \\
&= \frac{q_{S_t}}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)}. && \text{By definition of } p_{S_t} \text{ in Equation (2.27)}
\end{aligned}$$

Since LHS of Equation (2.30) sums over all possible S_t is one: $\sum_{S_t} \mathbb{P}(S_t \mid \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i) = 1$. Thus, summing over S_t for the RHS of Equation (2.30), we have:

$$(2.30) \quad 1 = \sum_{S_t} \frac{q_{S_t}}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)} = \frac{\sum_{S_t} q_{S_t}}{\mathbb{P} \left(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i \right)}$$

In the second equality, the reason that we can move the summation operator to the numerator is that the denominator is a constant *w.r.t.* all possible S_t . To be specific, given $S_t \subseteq \Gamma(S_{t-1})$, we have S_t is independent to $\mathcal{C} \setminus \Gamma(S_{t-1})$. Based on Equation (2.30), we finally obtain:

$$\sum_{S_t} q_{S_t} = \mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i).$$

The proof is finished. □

Lemma 5. *The probability of observing the sampling record $S_1, \dots, S_T$ by Algorithm 1 under Condition 1 is:*

$$(2.31) \quad \mathbb{P}(S_1, \dots, S_T) = q_{S_T} \prod_{t=1}^{T-1} p_{S_t}$$

PROOF. Given sampling record $S_1, \dots, S_{t-1}$, the conditional probability of observing the next record S_t, S'_t can be expanded based on Lemma 1,

$$\frac{\mathbb{P}(S_t|S_1, \dots, S_{t-1})}{\mathbb{P}(S'_t|S_1, \dots, S_{t-1})} = \frac{\mathbb{P}(S_t| \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)}{\mathbb{P}(S'_t| \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)}$$

Based on Equation (2.30), we can simplify the RHS of the above ratio equality and obtain:

$$\frac{\mathbb{P}(S_t|S_1, \dots, S_{t-1})}{\mathbb{P}(S'_t|S_1, \dots, S_{t-1})} = \frac{q_{S_t}}{q_{S'_t}}$$

Because of $\sum_{S_t} \mathbb{P}(S_t|S_1, \dots, S_{t-1}) = 1$ and Equation (2.30), we can get:

$$(2.32) \quad \mathbb{P}(S_t|S_1, \dots, S_{t-1}) = \frac{q_{S_t}}{\mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)}$$

We finally compute the probability of observing the sampling record $S_1 \dots, S_T$ by:

$$\begin{aligned} \mathbb{P}(S_1 \dots, S_T) &= \mathbb{P}(S_1) \prod_{t=2}^T \mathbb{P}(S_t|S_1, \dots, S_{t-1}) && \text{By Chain rule} \\ &= q_{S_1} \prod_{t=2}^T \frac{q_{S_t}}{\mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)} && \text{By Equation (2.32)} \\ &= q_{S_T} \prod_{t=2}^T \frac{q_{S_{t-1}}}{\mathbb{P}(\bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)} && \text{Left shift the numerator from } S_t \text{ to } S_{t-1} \\ &= q_{S_T} \prod_{t=1}^{T-1} p_{S_t} && \text{Plugin Lemma 3} \end{aligned}$$

The proof is finished. $\square$

2.8.2. An Upper Bound on Expected Running Time

Suppose the expected number of samplings of constraints c_i is $\mathbb{E}(T_i)$, then the total running time will be:

$$\mathbb{E}(T) \leq \sum_{i=1}^n \mathbb{E}(T_i)$$

Since each random variable has equal status, then the question comes down to the computation of individual T_i 's expectation. Let $S_1, \dots, S_T$ be any record of the algorithm that successfully terminates, and $T_i(S_1, \dots, S_T)$ be the total number of sampling related to constraint c_i throughout this record. Based on Lemma 5, we have:

$$\mathbb{E}(T_i) = \sum_{S_1, \dots, S_T} \mathbb{P}(S_1, \dots, S_T) T_i(S_1, \dots, S_T)$$

By far, we have shown the original proof of our work. We leave the difference between our proof with the existing one in section 2.8.3.

The rest of the computation can be done in the same way as the proof in [Guo et al., 2019]. Thus we cite the necessary intermediate steps in the existing work and finish the proof logic for the coherence of the whole running time analysis.

Lemma ([Guo et al., 2019] Lemma 12). *Let $q_\emptyset$ be a non-zero probability of all the constraints are satisfied. Let $q_{\{c_j\}}$ denote the probability that only constraint c_j is broken and the rest all hold. If $q_\emptyset > 0$, then $\mathbb{E}(T_i) = q_{\{c_j\}}/q_\emptyset$.*

After incorporating our fix, we can conclude the upper bound on the expected running time in Theorem 2.

Theorem 2 ([Guo et al., 2019] Theorem 13). *Under Condition 1, the total number of re-samplings throughout the algorithm is then $\frac{1}{q_0} \sum_{j=1}^L q_{\{c_j\}}$.*

2.8.3. Difference to the Existing Proof

The main difference in the above proof to the existing proof in [Guo et al., 2019, Theorem 13] is that: based on Equation (2.32) and (2.30), we show

$$\mathbb{P}(S_t | S_1, \dots, S_{t-1}) = \mathbb{P}(S_t | \bigwedge_{c_i \in \mathcal{C} \setminus \Gamma(S_{t-1})} c_i)$$

In [Guo et al., 2019]’s Equation (9), the first step cannot hold without the above equality. The original paper uses this result directly without providing enough justification.

2.9. Constrained MRF Model

2.9.1. Single Variable Form of Constrained MRF

Here we provide an example of transforming MRF with pairwise and single potential functions into a single potential form by introducing extra variables. Given random variables X_1, X_2, X_3 , we have the following example MRF model:

$$\begin{aligned} \phi_\theta(x_1, x_2, x_3) &= \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 \\ P_\theta(x) &= \frac{\exp(\phi_\theta(x_1, x_2, x_3))}{Z(\theta)} \end{aligned}$$

In the above formula, we have a cross term $x_1 x_2$. Two Boolean variables can have 4 different assignments in total. Therefore we can construct 4 extra Boolean variables to encode all these assignments. To illustrate, we introduce extra random variables $\hat{X}_{00}$,

X_1	X_2	$\hat{X}_{00}, \hat{X}_{01}, \hat{X}_{10}, \hat{X}_{11}$
0	0	1, 0, 0, 0
0	1	0, 1, 0, 0
1	0	0, 0, 1, 0
1	1	0, 0, 0, 1

Table 2.5. 4 constraints for converting pairwise terms in the potential function into single variable form.

$\hat{X}_{01}, \hat{X}_{10}, \hat{X}_{11}$. We further introduce extra constraints: When $X_1 = 0, X_2 = 0$, the extra variable must take values: $\hat{X}_{00} = 1, \hat{X}_{01} = 0, \hat{X}_{10} = 0, \hat{X}_{11} = 0$. See the rest constraints in Table 2.5.

Then the new potential function, including extended variables and pairwise to single variable constraints $\mathcal{C}$, is reformulated as:

$$\begin{aligned}\hat{\phi}_\theta(x_1, x_2, x_3, \hat{x}_{00}, \hat{x}_{01}, \hat{x}_{10}, \hat{x}_{11}) &= \theta_1 x_1 + \theta_2 x_2 + \theta_3 \hat{x}_{00} + \theta_3 \hat{x}_{01} + \theta_3 \hat{x}_{10} + \theta_3 \hat{x}_{11} \\ P_\theta(x|\mathcal{C}) &= \frac{\exp(\hat{\phi}_\theta(x_1, x_2, x_3, \hat{x}_{00}, \hat{x}_{01}, \hat{x}_{10}, \hat{x}_{11}))}{Z_{\mathcal{C}}(\theta)}\end{aligned}$$

For clarity, the newly added constraints do not impact Condition 1. Since the single variable transformation in the MRFs model originates from [Sang et al., 2005], thus is not considered as our contribution.

2.9.2. Gradient of log-Partition Function $\nabla \log Z_{\mathcal{C}}(\theta)$

We use the Chain rule of the gradient to give a detailed deduction of Equation (2.4).

$$\begin{aligned}\nabla \log Z_{\mathcal{C}}(\theta) &= \frac{\nabla Z_{\mathcal{C}}(\theta)}{Z_{\mathcal{C}}(\theta)} = \frac{1}{Z_{\mathcal{C}}(\theta)} \nabla \sum_{x \in \mathcal{X}} \exp(\phi_\theta(x)) C(x) \\ &= \sum_{x \in \mathcal{X}} \frac{\exp(\phi_\theta(x)) C(x)}{Z_{\mathcal{C}}(\theta)} \nabla \phi_\theta(x) = \sum_{x \in \mathcal{X}} P_\theta(x|\mathcal{C}) \nabla \phi_\theta(x) = \mathbb{E}_{x \sim P_\theta(\tilde{x}|\mathcal{C})} (\nabla \phi_\theta(x).)\end{aligned}$$

The above result shows the gradient of the constrained partition function is equivalent to the expectation of the gradient of the potential function $\nabla\phi_\theta$ over the model’s distribution (*i.e.*, $P_\theta(\tilde{x}|\mathcal{C})$). Therefore, we transform the gradient estimation problem into the problem of sampling from the current MRF model.

2.10. Experiment Settings and Configurations

2.10.1. Implementation Details

Implementation of NELSON. The proposed sampler can be implemented with Numpy, Pytorch, or Jax. We further offer a “batch version” implementation, that draws a batch of samples in parallel on GPU. The batched sampler is useful for those tasks that require a huge number of samples to estimate the gradient with a small approximation error.

In Section 2.4.1, we define vector of assignment $x^t = (x_1^t, \dots, x_n^t)$, where x_i^t is the assignment of variable X_i in the t -th round of Algorithm 1. $x_i^t = 1$ denotes variable X_i takes value 1 (or true). In the batch version, we define the matrix for a batch of assignments. Let b be the batch size, we have

$$x^t = \begin{bmatrix} x_{11}^t & \dots & x_{1n}^t \\ \vdots & \ddots & \vdots \\ x_{b1}^t & \dots & x_{bn}^t \end{bmatrix}$$

In the following part, we provide the detailed computation pipeline for the batch version of the proposed algorithm.

Initialization. The first step is to sample an initial assignment of X from the given the marginal probability vector P :

$$(2.33) \quad x_{li}^1 = \begin{cases} 1 & \text{if } u_{li} > P_i, \\ 0 & \text{otherwise.} \end{cases}, \quad \text{if } 1 \leq i \leq n, 1 \leq l \leq b$$

Here u_{li} is sampled from the uniform distribution over $[0, 1]$.

Check Constraint Satisfaction. The second step extract which constraint is violated. Given an assignment x^t at round $t \geq 1$, tensor W and matrix b , the computation of tensor Z^t is:

$$Z_{lik}^t = \sum_{i=1}^n W_{ikj} x_{lj}^t + b_{lik},$$

The special multiplication between tensor and matrix can be efficiently implemented with the Einstein summation³. Note that $Z_{ljk}^t = 1$ indicates for l -th batched assignment x_l , the k -th literal of j -th clause is true (takes value 1). Next, we compute S_{lj}^t as:

$$S_{lj}^t = 1 - \max_{1 \leq k \leq K} Z_{ljk}^t, \quad \text{for } 1 \leq j \leq L, 1 \leq l \leq b$$

Here $S_{lj}^t = 1$ indicates x_l^t violates j -th clause. We can check $\sum_{l=1}^b \sum_{j=1}^L S_{lj}^t \neq 0$ to see if any clause is violated for the current batch of assignments, which corresponds to $\sum_{i=1}^b C(x_l) = 0$.

Extract Variables in Violated Clauses. We extract all the variables that require resampling based on vector S^t computed from the last step. The vector of the resampling

³https://github.com/dgasmith/opt_einsum

indicator matrix A^t can be computed as:

$$A_{li}^t = \mathbf{1} \left(\sum_{j=1}^L S_{lj}^t V_{ji} \geq 1 \right), \quad \text{for } 1 \leq i \leq n, 1 \leq l \leq b$$

where $\sum_{j=1}^L S_{lj}^t V_{ji} \geq 1$ implies X_{li} requires resampling.

Resample. Given the marginal probability vector P , resample indicator matrix A^t and assignment matrix x^t , we draw a new random sample x^{t+1} .

$$x_{li}^{t+1} = \begin{cases} (1 - A_{li}^t)x_{li}^t + A_{li}^t & \text{if } u_{li} > P_i, \\ (1 - A_{li}^t)x_{li}^t & \text{otherwise.} \end{cases} \quad \text{for } 1 \leq i \leq n, 1 \leq l \leq b$$

where u_{li} is drawn from the uniform distribution in $[0, 1]$.

Since GPUs are more efficient at computing tensor, matrix, and vector operations but are slow at processing for loops. Drawing a batch of samples using the above extended computational pipeline is much faster than using a for loop over the computational pipeline in Section 2.4.1.

The sampler is involved with one hyper-parameter T_{tryout} . NELSON would terminate when it reaches T_{tryout} number of re-samples. This practice is commonly used to handle randomized programs that might run forever in rare cases.

Implementation of Algorithm 2. We first use the Constraints $\mathcal{C}$ and parameters θ^t to build the current NELSON module. Then we draw m samples from NELSON module $\{\tilde{x}^j\}_{j=1}^m$ and draw from dataset randomly m samples $\{x^j\}_{j=1}^m$. Continuing from that point, we compute the potential value from the two sets of inputs, i.e., $\{\phi_\theta(\tilde{x}^j)\}_{j=1}^m$ and $\{\phi_\theta(x^j)\}_{j=1}^m$. Pytorch would be slow if we compute each potential's gradient using a

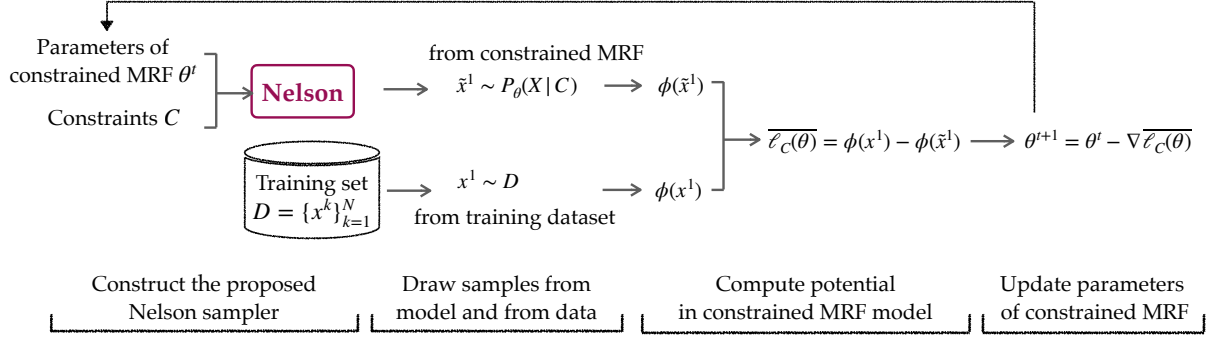


Figure 2.4. Implementation pipeline of the NELSON-CD algorithm with $m = 1$. The proposed NELSON can be efficiently adapted to a Pytorch-based machine learning library and enforces constraint satisfaction during learning.

for-loop. To bypass this problem, we instead compute the following:

$$(2.34) \quad \overline{\ell_C(\theta)} = \frac{1}{m} \sum_{j=1}^m \phi_\theta(x^j) - \frac{1}{m} \sum_{j=1}^m \phi_\theta(\tilde{x}^j).$$

Following that, we call the PyTorch library's gradient function, which computes exactly

$$\nabla \overline{\ell_C(\theta)} = \nabla \left(\frac{1}{m} \sum_{j=1}^m \phi_\theta(x^j) - \frac{1}{m} \sum_{j=1}^m \phi_\theta(\tilde{x}^j) \right) = \frac{1}{m} \sum_{j=1}^m \nabla \phi_\theta(x^j) - \frac{1}{m} \sum_{j=1}^m \nabla \phi_\theta(\tilde{x}^j)$$

Note that $\nabla \overline{\ell_C(\theta)}$ recovers the result in Equation (2.4). Finally, we update the parameters θ . The proposed NELSON module and the neural network are computed on the same GPU device. This allows us to exploit the parallel computing power of modern GPUs and remove time for the data transfer from CPU to GPU or vice versa. See Figure 2.4 for a visualized overview of the implementation with Pytorch.

2.10.2. Learn Random K-SAT Solutions with Preference

Task Definition. We are given a training set $\mathcal{D}$ containing some preferred assignments $\mathcal{D} = \{x^j\}_{j=1}^N$ for the corresponding CNF formula $c_1 \wedge \dots \wedge c_L$. We require the CNF formula to be true. This means, by the definition of CNF formulas, that every clause has to be satisfied. These clauses become our set of constraints. Under the constrained MRF model, the learning task is to maximize the log-likelihood of the assignments seen in the training set $\mathcal{D}$. The inference task is to generate valid solutions from the learned model’s distribution [Dodaro and Previti, 2019; Rosa et al., 2011].

Dataset. We denote the Boolean variables’ size in K -SAT as the “problem size”. We consider several datasets of different problem sizes generated from CNFGen⁴ [Lauria et al., 2017] random K -SAT functions. K is fixed as 5; the number of variables and clauses are kept the same, ranging from 10 to 1500. We generate 100 different CNF formulas for every problem size. To generate the training set $\mathcal{D}$, we use the Glucose4 solver from PySAT⁵ library [Ignatiev et al., 2018] to generate 200 assignments randomly as the preferred assignments for every formula.

It should be pointed out that we don’t consider datasets like SATLIB and SAT competitions. It is mainly because these datasets are hard instances with a much larger input space but a limited number of solutions. NELSON would generally take exponential time to find these solutions, just like finding needles in a haystack. The other reasons are that using neural networks to learn these limited assignments is straightforward since we can simply hard-wire the network to memorize all the valid assignments. The main purpose

⁴<https://github.com/MassimoLauria/cnfgen>

⁵<https://pysathq.github.io/>

of this work is to let a constrained MRF learn a representation for the underlying preference pattern, not create a neural solver that can generate valid assignments for any CNF formula. Thus, we conform to the settings of the easy formula where obtaining valid solutions is easy.

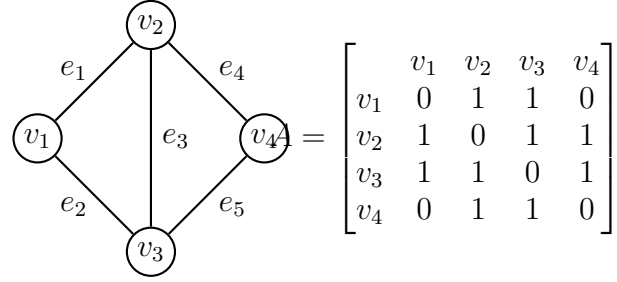
2.10.3. Learn Sink-Free Orientation in Undirected Graphs

Task Definition In graph theory, a *sink-free* orientation of an undirected graph is a choice of orientation for each edge such that every vertex has at least one outgoing edge [Cohn et al., 2002]. It has wide applications in robotics routing and IoT network configuration [Takahashi et al., 2009]. The Constraints for this problem are that every vertex has at least one outgoing edge after orientation. As stated in [Guo et al., 2019], these constraints satisfy Condition 1.

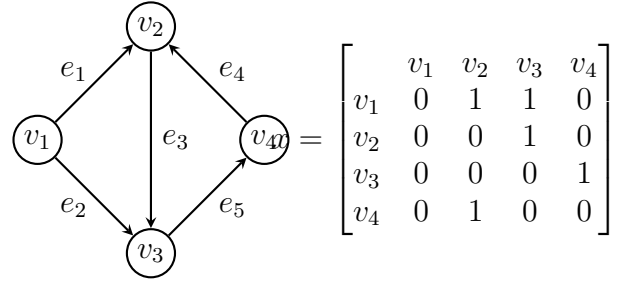
See Figure 2.5 for an example graph and one possible sink-free edge orientation. We define binary variables $X_1, \dots, X_m$, and associate variable X_i to edge e_i for $1 \leq i \leq m$. Variable X_i takes value 1 if the edge orientation is $v_i \rightarrow v_j$ where $i < j$. Otherwise, X_i takes value 0. The constraints are:

$$\mathcal{C} = (X_1 \vee X_2) \wedge (\neg X_1 \vee X_3 \vee \neg X_4) \wedge (\neg X_2 \vee \neg X_3 \vee X_5) \wedge (X_4 \vee \neg X_5)$$

where the single constraint $c_1 = (X_1 \vee X_2)$ corresponds to vertex v_1 , constraint $c_2 = (\neg X_1 \vee X_3 \vee \neg X_4)$ corresponds to vertex v_2 , constraint $c_3 = (\neg X_2 \vee \neg X_3 \vee X_5)$ corresponds to vertex v_3 , and constraint $c_4 = (X_4 \vee \neg X_5)$ corresponds to vertex v_4 . The orientation assignment matrix x shown in Figure 2.5(b) implies: $X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 0, X_5 = 1$.



(a) An undirected graph G with its adjacency matrix A



(b) An orientation of the edges and the orientation matrix x

Figure 2.5. **(a)** An un-directed graph $G(V, E)$ where the vertices are $V = \{v_1, v_2, v_3, v_4\}$ and the un-directed edges are $E = \{e_1 = (v_1, v_2), e_2 = (v_1, v_3), e_3 = (v_2, v_3), e_4 = (v_2, v_4), e_5 = (v_3, v_4)\}$. **(b)** A possible sink-free orientation of the edges in the graph and its matrix representation x , where every vertex has at least one outgoing edge.

Notations. Let graph $G(V, E)$ be an un-directed graph; its adjacency matrix A that represents graph connectivity is:

$$(2.35) \quad A_{ij} = \begin{cases} 1 & \text{If } (v_i, v_j) \in E \\ 0 & \text{otherwise} \end{cases}$$

A possible assignment for the orientation of every edge can be represented as a matrix $x \in \{0, 1\}^{|V| \times |V|}$:

$$(2.36) \quad x_{ij} = \begin{cases} 1 & \text{if the edge orientation is } v_i \rightarrow v_j \\ 0 & \text{otherwise} \end{cases}$$

In the constrained MRF model defined in Eq. (2.6), the potential function of one orientation of all edges is

$$\phi_\theta(x) = \sum_{i=1}^{|V|} \sum_{j=1}^{|V|} \theta_{ij} A_{ij} x_{ij}$$

A single constraint for vertex v_k is $c_k(x) = \mathbf{1} \left(\sum_{j=1}^n A_{k,j} x_{k,j} = 1 \right)$. If there is no ongoing edge of vertex v_k . The constraint function $C(x)$ is defined as: $\prod_{i=1}^n c_k(x)$. In Algorithm 1 step 1, edge (v_i, v_j) will pick the orientation $v_i \rightarrow v_j$ with probability:

$$\frac{\exp(\theta_{ij} A_{ij} x_{ij})}{\exp(\theta_{ji} A_{ji} x_{ji}) + \exp(\theta_{ij} A_{ij} x_{ij})}$$

Dataset. We use the NetworkX⁶ package to generate random Erdos Renyi graph with edge probability 0.55. The problem size refers to the number of vertices in the graph, we range the problem size from 10 to 100. For each problem size, we generate 100 different random undirected graphs. We then convert the graph into CNF form using the above edge-variable conversion rule. Afterward, we follow the same processing steps as the previous problem that learn preferential solution distribution for random K-SAT.

⁶<https://networkx.org/>

2.10.4. Learn Vehicle Delivery Routes

Given a set of locations to visit, the task is to generate a sequence to visit these locations in which each location is visited once and only once and the sequence closely resembles the trend presented in the training data. The training data are such routes collected in the past. The dataset is constructed from TSPLIB, which consists of 29 cities in Bavaria, Germany. In Figure 2.3, we see NELSON can obtain samples of this delivery problem highly efficiently.

A possible travel plan can be represented as a matrix $x \in \{0, 1\}^{|V| \times |V|}$:

$$(2.37) \quad x_{ij} = \begin{cases} 1 & \text{if edge } v_i \rightarrow v_j \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

The constraints are that every routing plan should visit every location once and only once.

Similarly, in the constrained MRF model defined in Eq. (2.6), the potential function of the vehicle routing plan is

$$\phi_{\theta}(x) = \sum_{i=1}^{|V|} \sum_{j=1}^{|V|} \theta_{ij} A_{ij} x_{ij}$$

2.10.5. Detailed Baselines Configuration

In terms of sampling-based methods, we consider:

- Gibbs sampler [Carter and Kohn, 1994], a special case of MCMC that is widely used in training MRF models. In each step, the Gibbs algorithm samples one

dimension based on a conditional marginal distribution. We follow this implementation⁷.

- Weighted SAT samplers, including WAPS⁸ [Gupta et al., 2019], WeightGen⁹, [Chakraborty et al., 2014] and XOR sampler¹⁰ [Ermon et al., 2013; Ding and Xue, 2021].
- Uniform SAT samplers, including UniGen¹¹, [Soos et al., 2020], QuickSampler¹², [Dutra et al., 2018], CMSGen¹³ [Golia et al., 2021] and KUS¹⁴ [Sharma et al., 2018].

Currently, there are only GPU-based SAT solvers [Prevot et al., 2021; Mahmoud, 2022] and model counters [Fichte et al., 2019], GPU-based SAT samplers are not available by far.

2.10.6. Detailed Definition of Evaluation Metrics

In terms of evaluation metrics, we consider

- Training time per epoch. The average time for the whole learning method to finish one epoch with each sampler.
- Validness. The learned model is adopted to generate assignments and we evaluate the percentage of generated assignments that satisfy the constraints.

⁷<https://github.com/Fading0924/BPChain-CD/blob/master/mrf.py>

⁸<https://github.com/meelgroup/waps>

⁹<https://bitbucket.org/kuldeepmeel/weightgen/src/master/>

¹⁰<https://cs.stanford.edu/~ermon/code/srcPAWS.zip>

¹¹<https://github.com/meelgroup/unigen>

¹²<https://github.com/RafaelTupynamba/quicksampler>

¹³<https://github.com/meelgroup/cmsgen>

¹⁴<https://github.com/meelgroup/KUS>

- Mean Averaged Precision (MAP@10). This is a ranking-oriented metric that can evaluate the closeness of the learned MRF distribution to the goal distribution. If the model learns the goal distribution in the training set, then it would assign a higher potential value to those assignments in the training set than all the rest unseen assignments. Based on this principle, we randomly pick two sets of inputs in those valid assignments: seen assignments from the training set and unseen assignments that are randomly generated. We use the value of factor potential $\phi(x)$ to rank those assignments in ascending order. Next, we check how many preferred solutions can fall into the Top-10 by computing the following

$$\text{MAP@10} = \sum_{k=1}^{10} \frac{\#\text{preferred assignments among top-}k}{k}$$

- log-likelihood of assignments in the training set $\mathcal{D}$. The model that attains the highest log-likelihood learns the closest distribution to the training set. Specifically, given a training set $\mathcal{D} = \{x^k\}_{k=1}^N$ and parameters θ , the log-likelihood value is:

$$(2.38) \quad \frac{1}{N} \sum_{k=1}^N \log P_{\theta}(X = x^k | \mathcal{C}) = \frac{1}{N} \sum_{k=1}^N \phi_{\theta}(x^k) - \log Z_{\mathcal{C}}(\theta)$$

We use the ACE algorithm to compute the approximated value of $\log Z_{\mathcal{C}}(\theta)$ ¹⁵.

- Approximation Error of $\nabla \log Z_{\mathcal{C}}(\theta)$, that is the L_1 distance between the exact gradient of $\log Z_{\mathcal{C}}(\theta)$ in Eq. (2.4) and the empirical gradient from the sampler. For small problem sizes, we enumerate all $x \in \mathcal{X}$ to get the exact gradient and

¹⁵<http://reasoning.cs.ucla.edu/ace/moreInformation.html>

draw samples $\{\tilde{x}^j\}_{j=1}^m$ with $m = 2000$ from every sampler for approximation.

$$\left| \underbrace{\sum_{x \in \mathcal{X}} \frac{\exp\left(\sum_{j=1}^n \theta_j x_j\right) C(x)}{Z_C(\theta)} x_i}_{\text{Exact gradient term}} - \underbrace{\sum_{j=1}^m \tilde{x}_i^j}_{\text{Estimated gradient with sampler}} \right|$$

For fixed parameter θ , the best sampler would attain the smallest approximation error.

2.10.7. Hyper-parameter Settings

In the implementation of NELSON, we set the maximum tryout of resampling as $T_{tryout} = 1000$ for all the experiments and all the datasets.

For the hyper-parameters used in learning the constrained MRF, we set the number of samples from the model to be $m = 200$, the learning rate η is configured as 0.1 and the total learning iterations are $T_{\max} = 1000$.

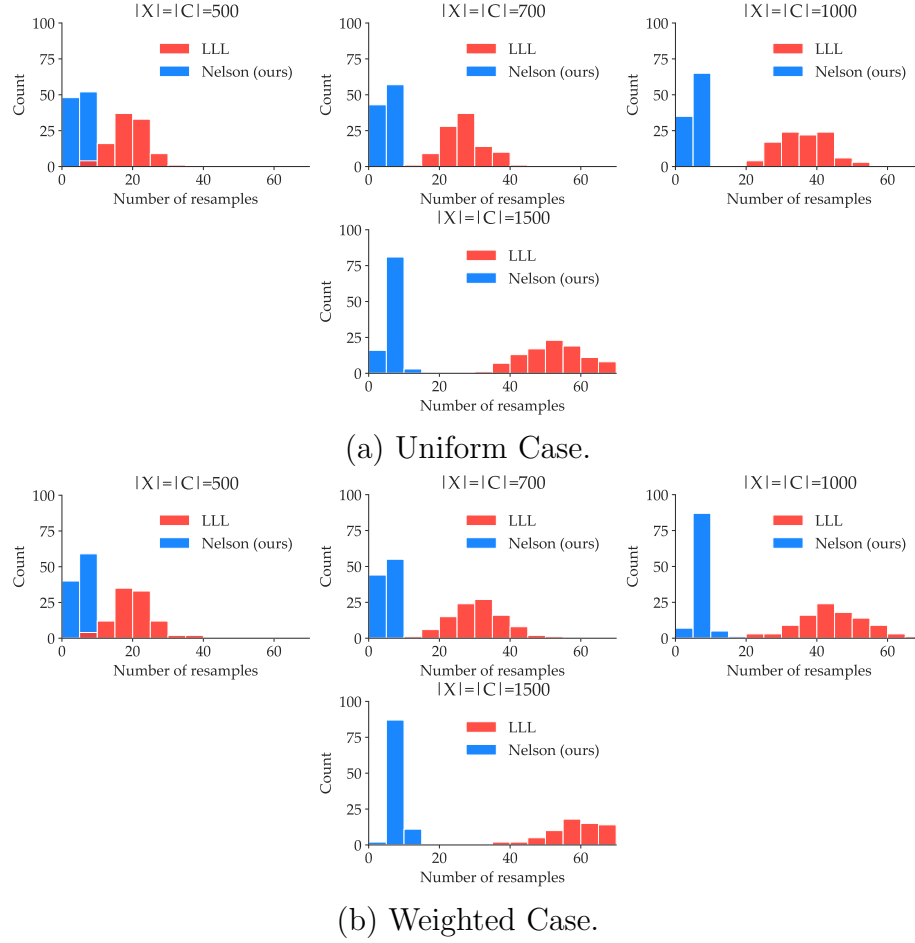


Figure 2.6. The distribution of resampling steps in the NELSON and Algorithmic-LLL [Moser and Tardos, 2010]. Both of them get a valid sample within $T_{tryouts}$. NELSON takes much fewer resamples than Algorithmic-LLL because it resamples all the violated clauses at every iteration while Algorithmic-LLL only resamples one of them.

CHAPTER 3

VarFA: A Variational Factor Analysis Framework for Efficient Bayesian Learning Analytics

Overview

In the last chapter of this thesis, we propose VarFA, a variational inference factor analysis framework that extends existing factor analysis models for educational data mining to efficiently output uncertainty estimation in the model’s estimated factors. Such uncertainty information is useful, for example, for an adaptive testing scenario, where additional tests can be administered if the model is not quite certain about a students’ skill level estimation. Traditional Bayesian inference methods that produce such uncertainty information are computationally expensive and do not scale to large data sets. VarFA utilizes variational inference which makes it possible to efficiently perform Bayesian inference even on very large data sets. We use the sparse factor analysis model as a case study and demonstrate the efficacy of VarFA on both synthetic and real data sets. VarFA is also very general and can be applied to a wide array of factor analysis models. Code and instructions to reproduce results in this chapter are available from <https://tinyurl.com/tvm4332>.

3.1. Introduction

A core task for many practical educational systems is *student modeling*, i.e., estimating students' mastery level on a set of skills or knowledge components (KC) VanLehn [1988]; Chrysafiadi and Virvou [2013]. Such estimates allow in-depth understanding of students' learning status and form the foundation for automatic, intelligent learning interventions. For example, intelligent tutoring systems (ITSs) Shute and Psotka [1994]; Graf and Kinshuk [2012]; Anderson et al. [1995]; Nye et al. [2014]; Heffernan and Heffernan [2014]; Rafferty et al. [2015] rely on knowing the students' skill levels in order to effectively recommend individualized learning curriculum and improve students' learning outcomes. In the big data era, student modeling is usually formulated as an educational data mining (EDM) problem Romero and Ventura [2010]; Baker and Yacef [2009]; Merceron and Yacef [2005] where an underlying machine learning (ML) model estimates students' skill mastery levels from students' learning records, i.e., their answers to assessment questions.

Many student modeling methods have been proposed in prior literature. A fruitful line of research for student modeling follows the *factor analysis (FA)* approach. FA models usually assume that an unknown, potentially multi-dimensional student parameter, in which each dimension is associated with a certain skill, explains how a student answers questions and is to be estimated. Popular and successful FA models include item response theory (IRT) van der Linden and Hambleton [2013], multi-dimensional IRT Ackerman [1994], learning factor analysis (LFA) Cen et al. [2006], performance factor analysis (PFA) Pavlik et al. [2009], DASH (short for difficulty, ability, and student history) Lindsey et al. [2014]; Mozer and Lindsey [2016], DAS3H (short for difficulty, ability, skill and student skill history) Choffin et al. [2019], knowledge tracing machines (KTM) Vie and

Kashima [2019], and so on. Recently, more complex student models based on deep neural networks (DNN) have also been proposed Piech et al. [2015]; Zhang et al. [2017]; Minn et al. [2018]. Nevertheless, thanks to their simplicity, effectiveness and robustness, FA models remain widely adopted and investigated for practical EDM tasks. Moreover, there is evidence that simple FA models could even outperform DNN models for student modeling in terms of predicting students' answers Wilson et al. [2016]. Because of FA models' competitive performance and its elegant mathematical form, we focus on FA-based student models in this chapter.

Most of the aforementioned FA models compute a single point estimate of skill levels for each student. Often, however, it is not enough to obtain mere point estimates of students' skill levels; knowing the model's uncertainty in its estimation is crucial because it potentially helps improve the model's performance and improve both students' and instructors' experience with educational systems. For example, in ITS, an underlying model can use the uncertainty information in its estimation of students' skill level to automatically decide that its recommendations based on highly uncertain estimations are unreliable and instead notify a human instructor to evaluate the students' performance. This enables collaboration between ITS and instructors to create a more effective learning environment. In adaptive testing systems Chang et al. [2009]; Yan et al. [2016], knowing the uncertainty in model's estimation could help the model intelligently pick the next test items to most effectively reduce its uncertainty about estimated students' skill levels. This will help to potentially reduce the number of items needed to have a confident, accurate estimation of the students' skill mastery level, saving time for both students to take the test and instructors to have a good assessment of the student's skills.

All the above applications require the model to “know what it does not know,” i.e., to quantify the uncertainty of its estimation. Achieving this does not necessarily change the model. rather, we need a different inference algorithm for inferring not only a point estimate of student’s skill level from observed data (i.e., students’ answer records to questions) but also uncertainty in the estimations.

Fortunately, there exist methods that both compute point estimates of students’ skill levels and quantifies the uncertainty of those estimates. These methods usually follow the Bayesian inference paradigm, where each student’s unknown skill levels are treated as random variables drawn from a *posterior distribution*. Thus, one can use the *credible interval* to quantify the model’s uncertainty on the student’s estimated skill level. A classical method for Bayesian inference is Monte Carlo Markov Chain (MCMC) sampling [Gelman et al. \[2013\]](#) which has been widely used in many other disciplines [Gilks et al. \[1995\]](#) other than EDM. In the context of student modeling with FA models, existing works such as [Lan et al. \[2014\]](#); [Fox \[2010\]](#); [Pardos et al. \[2010\]](#); [Merkle \[2011\]](#) have applied MCMC methods to obtain credible intervals.

Unfortunately, classic Bayesian inference methods suffer from extensive computational complexity. For example, each computation step in MCMC involves a time-consuming evaluation of the posterior distribution. Making matters worse, MCMC typically takes many more steps to converge than non-Bayesian inference methods. As a concrete illustration, in [Lan et al. \[2014\]](#), the Bayesian inference method (10 minutes) is about 100 times slower than the other non-Bayesian inference method based on stochastic gradient descent (6 seconds). The high computational cost prevents Bayesian inference methods

from mass-deployment in large-scale, real-time educational systems where timely feedback is critical for learning Driscoll [1994] and tens of thousands of data points need to be processed in seconds or less instead of minutes or hours. It is thus highly desirable to accelerate Bayesian inference so that one can quantify model’s uncertainty in its estimation as efficiently as non-Bayesian inference methods.

Contributions. In this work, we propose VarFA, a novel framework based on *variational inference* (VI) to perform efficient, scalable Bayesian inference for FA models. The key idea is to approximate the true posterior distribution, whose costly computation slows down Bayesian inference, with a *variational distribution*. We will see in Section 3.3 that, with this approximation, we turn Bayesian inference into an optimization problem where we can use the same efficient inference algorithms as in non-Bayesian inference methods. Moreover, the variational distribution is very flexible and we have full control specifying it, allowing us to freely use the latest development in machine learning, e.g., deep neural networks (DNNs), to design the variational distribution that closely approximates the true posterior. Thus, we also regard our work as a first step in applying DNNs to FA models for student modeling, achieving efficient Bayesian inference (enabled by DNNs) without losing interpretability (brought by FA models). We demonstrate the efficacy of our framework on both synthetic and real data sets, showcasing that VarFA substantially accelerates classic Bayesian inference for FA models with no compromise on performance.

The remainder of this chapter is organized as follows. Section 3.2 introduces the problem setup in FA and reviews the important FA models and their inference methods, in particular Bayesian inference. Section 3.3 explains our VarFA framework in detail. Section 3.4 presents extensive experimental results that substantiate the claimed advantages

of our framework. Section 3.5 concludes this chapter and discusses possible extensions to VarFA.

3.2. Background and Related Work

We first set up the problem and review related work. Assume we have a data set $\mathbf{Y} \in \mathbb{R}^{N \times Q}$ organized in matrix format where N is the total number of students and Q is the number of questions. This is a binary students' answer record matrix where each entry y_{ij} represents whether student i correctly answered question j . Usually, not all students answer all questions. Thus, $\mathbf{Y}$ contains missing values. We use $\{i, j\} \in \Omega_{\text{obs}}$ to denote entries in $\mathbf{Y}$, i.e., the i -th student's answer record to the j -th question, that are observed.

We are interested in models capable of inferring each i -th student's skill mastery level that can accurately predict the student's answers given the above data. These models are often evaluated on the prediction accuracy and whether the inferred student skill mastery levels are easily interpretable and educationally meaningful. We now review factor analysis models (FA), one of the most widely adopted and successful methodologies for the student modeling task.

3.2.1. Factor Analysis For Student Modeling

One of the earliest FA model for student modeling is based on the item response theory (IRT) [van der Linden and Hambleton \[2013\]](#). It usually has the following form

$$(3.1) \quad \mathbb{P}(y_{ij} = 1) = \sigma(c_i + \mu_j),$$

which assumes that each student's answer y_{ij} is independently Bernoulli distributed. The above formula says that the students' answers can be explained by an unknown scalar student skill level factor c_i for each student i and an unknown scalar question difficulty level factor μ_j for each question j . $\sigma(\cdot)$ is a Sigmoid activation function, i.e., $\sigma(x) = 1/(1 + \exp(-x))$. The multi-dimensional IRT model (MIRT) [Ackerman \[1994\]](#) extends IRT by using a multi-dimensional vector to represent the student skill levels. More recently, [Lan et al. \[2014\]](#) proposed sparse factor analysis model (SPARFA) which extends MIRT by imposing additional assumptions, resulting in improved interpretations of the inferred factors.

Other FA models seek to improve student modeling performance by cleverly incorporate additional auxiliary information. For example, the additive factor model (AFM) [Cen et al. \[2006\]](#) incorporates students' accumulative correct answers for a question and the skill tags associated with each question:

$$(3.2) \quad \mathbb{P}(y_{ij} = 1) = \sigma \left(c_i + \sum_{k=1}^{\kappa} (q_{kj}\beta_k + q_{kj}\rho_k b_{ik}) \right),$$

where κ is the total number of skills in the data, b_{ik} is the i -th student's total number of correct answers for skill k and $q_{kj} \in \{0, 1\}$ indicates whether the skill k is associated with question j . In particular, q_{kj} 's form matrix $\mathbf{Q} \in \mathbb{R}^{K \times Q}$ which is commonly known as the Q-matrix in literature [Tatsuoka \[1983\]](#). The unknown, to-be-inferred factors are β_k , a scalar difficulty factor for each skill k , and ρ_k , a scalar learning rate of skill k . The performance factor model (PFM) [Pavlik et al. \[2009\]](#) builds upon AFM that additionally incorporate the total number of a student's incorrect answers to questions associated with a skill. The instructor factor model (IFM) [Chi et al. \[2011\]](#) further builds on PFM

to incorporate the prior knowledge of whether a student has already mastered a skill. More recently, [Vie and Kashima \[2019\]](#) introduces knowledge tracing machines (KTM) that could flexibly incorporate a number of auxiliary information mentioned above, thus generalizing AFM, PFM and IFM.

Another way to improve student modeling performance is to utilize the so-called *memory*, i.e., using students' historic action data over time. For example, [Lindsey et al. \[2014\]](#); [Mozer and Lindsey \[2016\]](#) proposed DASH (short for difficulty, ability, and student history) that incorporates a student's total number of correct answers and total number of attempts for a question in a given time window

$$(3.3) \quad \mathbb{P}(y_{ij} = 1) = \sigma \left(c_i - \mu_j + \sum_{w=0}^{W-1} \left(\theta_{2w+1} \log(1 + \rho_{ijw}) - \theta_{2w+2} \log(1 + \gamma_{ijw}) \right) \right)$$

where W is the length of the time windows (e.g., number of days), ρ_{ijw} and γ_{ijw} are the i -th student's total number of correct answers and total number of attempts for question j at time w , respectively. θ 's are parameters that captures the effect of correct answers and total attempts. More recently, DAS3H [Choffin et al. \[2019\]](#) extends DASH to further include skill tags associated with each question which essentially amounts to adding, within the last summation term in Eq. 3.3, another summation over the number of skills.

3.2.1.1. A general formulation for FA models. Although the above FA models differ in their formulae, modeling assumptions and the available auxiliary data used, we argue that the aforementioned FA models can be unified into a canonical formulation below

$$(3.4) \quad \mathbb{P}(y_{ij} = 1) = \sigma(\mathbf{c}_i^\top \mathbf{m}_j + \mu_j),$$

where $\mathbf{c}_i \in \mathbb{R}^K$, $\mathbf{m}_j \in \mathbb{R}^K$ and $\mu_j \in \mathbb{R}$ are factors whose dimension, interpretations and subscript indices depend on the specific instantiations of the FA model. To illustrate that Eq. 3.4 subsumes the FA models mentioned above, we demonstrate, as examples, how to turn SPARFA and AFM into the form in Eq. 3.4 and how to reinterpret the parameters in the reformulation. The equivalence between Eq. 3.4 and the original SPARFA formula is immediate and we can interpret the parameters in Eq. 3.4 as follows to recover SPARFA: K is the number of *latent skills* that group the actual skill tags in the data set into meaningful coarse clusters; $\mathbf{c}_i$ represents the i -th student's skill level on the latent skills; $\mathbf{m}_j$ represents the strength of association of the j -th question with the latent skills which is assumed to be nonnegative and sparse for improved interpretation; and μ_j represents the difficulty of question j . All three factors in SPARFA are assumed to be unknown.

Similarly, for AFM, we can perform the following change of variables and indices to obtain Eq. 3.4. We first change the indices $\mathbf{m}_j$ to $\mathbf{m}_{ij}$ and μ_j to μ_i , and remove the index in $\mathbf{c}_i$ to $\mathbf{c}$. Then, we simply set $\mathbf{c} = [\beta_1, \dots, \beta_\kappa, \rho_1, \dots, \rho_\kappa]^\top$, $\mathbf{m}_{ij} = [q_{1j}, \dots, q_{\kappa j}, q_{1j}b_{ik}, \dots, q_{\kappa j}b_{ik}]^\top$ and $\mu_i = \alpha_i$ to obtain Eq. 3.4. We can interpret the parameters in the reformulation as follows to recover AFM: $\kappa = K/2$ is the total number of skill tags in the data; $\mathbf{c}$ represents the unknown skill information including skill difficulty and learning rate; $\mathbf{m}_{ij}$ summarizes known auxiliary information for the i -th student and j -th question; μ_i represents the unknown i -th student's skill mastery level. The other FA models can be cast into Eq. 3.4 in a similar fashion. Note that, if the observed data $\mathbf{Y}$ is not binary but rather continuous or categorical, we can simply use a Gaussian or categorical distribution for the observed data and change the Sigmoid activation $\sigma(\cdot)$ to some other activation functions accordingly. In

this way, we still retain the general FA model in Eq. 3.4. We will use this canonical form to facilitate discussions in the rest of this chapter.

3.2.2. Inference Methods for FA Models

We first introduce the inference objective and briefly review maximum log likelihood estimation method. Then, we review existing works that uses Bayesian inference for FA and highlight their high computational complexity, paving the way to VarFA, our proposed efficient Bayesian inference framework based on variational inference.

The factors in Eq. 3.4 may contain known factors that need no inference. Therefore, for convenience of notation, let $[\nu, \theta, \psi]$ be a partition of the factors $\mathbf{c}_i$, $\mathbf{m}_j$ and μ_j where ν contains the unknown students' skill level factors to be estimated, θ contains the remaining unknown factors and ψ contains the known factors. For example, for AFM, $\nu = \{\mu_1, \dots, \mu_N\}$, $\theta = \mathbf{c}$ and $\psi = \{\mathbf{m}_{11}, \dots, \mathbf{m}_{NQ}\}$. For SPARFA, $\nu = \{\mathbf{c}_1, \dots, \mathbf{c}_N\}$, $\theta = \{\mathbf{m}_1, \dots, \mathbf{m}_Q, \mu_1, \dots, \mu_Q\}$ and $\psi = \emptyset$. Further, let the subscripts for these partitions indicate the corresponding latent factors in FA; i.e., for SPARFA, $\theta_i = [\mathbf{m}_i, \mu_i]$ and $\nu_i = \mathbf{c}_i$. Since ψ summarizes known factors, we will omit it from the mathematical expositions in the remainder of the chapter.

The inference objective in FA is then to obtain a good estimate of the unknown factors, represented by θ and ν , with respect to some loss function $\mathcal{L}$, usually the marginal data log likelihood. There are two ways to infer the unknown factors.

3.2.2.1. Maximum Likelihood Estimation. The first way is through the maximum likelihood estimate (MLE) which obtains a point estimate of the unknown factors

$$(3.5) \quad \hat{\theta}, \hat{\nu} = \underset{\theta, \nu}{\operatorname{argmin}} \left(- \sum_{i, j \in \Omega_{\text{obs}}} \log p(y_{ij}; \theta, \nu) \right) + \lambda \mathcal{R}(\theta, \nu).$$

Recall that Ω_{obs} indicates which student i has provided an answer to question j . $\mathcal{R}(\theta, \nu)$ is a regularization term, e.g., ℓ_2 regularization on the factors and λ is a hyper-parameter that controls the strength of the regularization. Most of the works in FA use this method of inference [Cen et al. \[2006\]](#); [Pavlik et al. \[2009\]](#); [Chi et al. \[2011\]](#); [Vie and Kashima \[2019\]](#); [Choffin et al. \[2019\]](#); [Lindsey et al. \[2014\]](#); [Mozer and Lindsey \[2016\]](#). The advantage of MLE is that the above optimization objective allows the use of fast inference algorithms, in particular stochastic gradient descent (SGD) and its many variants, making the inference highly efficient.

3.2.2.2. Maximum A Posteriori Estimation. The second way is through maximum a posteriori (MAP) estimation through Bayesian inference. This is the focus of this chapter. Recall that we wish to obtain not only a point estimate but also credible interval, which MAP allows while MLE does not.

Bayesian inference methods treat the unknown parameters ν and θ as random variables drawn from some distribution. We are thus interested in inferring the *posterior* distribution of ν and θ . Using the Bayes rule, we have that

$$(3.6) \quad p(\nu | \mathbf{Y}, \theta) = \frac{p(\mathbf{Y}, \nu, \theta)}{p(\mathbf{Y})}$$

$$(3.7) \quad = \frac{p(\mathbf{Y} | \nu, \theta) p(\nu) p(\theta)}{\iint p(\mathbf{Y} | \nu, \theta) p(\nu) p(\theta) d\nu d\theta}.$$

We can similarly derive the posterior for the other unknown factor θ , although in this chapter we focus on Bayesian inference for the unknown student skill level parameter ν . In Eq. 3.6 to Eq. 3.7 we have applied standard Bayes rule. Note that, because ψ contains known factors and thus does not enter the Bayes rule equation, we have omitted it from the above equations. Once we estimate the posterior distribution for ν from data, we can obtain both point estimates and credible intervals by respectively taking the mean and the standard deviation of the posterior distributions. A number of prior works have proposed to use Bayesian inference for factor analysis, mostly developed for the IRT model Fox and Glas [2001]; Mislevy [1986]. More recently, Bayesian methods are developed for more complex models. For example, Lan et al. [2014] proposed SPARFA-B, a specialized MCMC algorithm that accounts for SPARFA’s sparsity and nonnegativity assumptions.

However, Bayesian inference suffers from high computational cost despite its desired capability to quantify uncertainty. The challenge stems from the difficulty to evaluate the denominator in the posterior distribution in Eq. 3.6 and 3.7 which involves an integration over a potentially multi-dimensional variable. This term usually cannot be computed in close form, thus we usually cannot get an analytical formula for the posterior distribution. Monte Carlo Markov Chain (MCMC) method gets around this difficulty by using analytically tractable proposal distributions to approximate the true posterior and sequentially updating the proposal distribution in a manner reminiscent of gradient descent. MCMC methods also enjoy the theoretical advantage that, when left running for long enough, the proposal distribution is guaranteed to converge to the true posterior distribution Gelman et al. [2013]. However, in practice, it might take very long time for MCMC to converge

to the point that running MCMC is no longer practical when data set is very large. The high computational complexity of MCMC is apparently impractical for educational applications where timely feedback is of critical importance [Driscoll \[1994\]](#).

3.3. VarFA: A Variational Inference Factor Analysis Framework

We are ready to introduce VarFA, our variational inference (VI) factor analysis framework for efficient Bayesian learning analytics. The core idea follows the variational principle, i.e., we use a parametric variational distribution to approximate the true posterior distribution. VarFA is highly flexible and efficient, making it suitable for large scale Bayesian inference for FA models in the context of educational data mining.

In this current work, we focus on obtaining credible interval for the student skill mastery factor ν as a first step of VarFA because, recall from [Section 3.1](#), this factor is often of more practical interest than the other unknown factors. Therefore, currently VarFA is a hybrid MAP and MLE inference method: we perform VI on the unknown student factor ν and MLE on all other unknown factors θ . We consider this hybrid nature of VarFA in its current formulation as a novel feature because classic MLE or MAP estimation are not capable of performing Bayesian inference on a subset of the unknown factors. Extension to VarFA to full Bayesian inference for all unknown factors is part of an ongoing research; see [3.5](#) for more discussions.

3.3.1. VarFA Details

Now, we explain in detail how to apply variational inference for FA models for efficient Bayesian inference. Because the posterior distribution is intractable to compute (recall

Eq. 3.6 and 3.7 and the related discussion), we approximate the true posterior distribution for ν with a parametric variational distribution

$$(3.8) \quad p(\nu|\mathbf{Y}, \theta) \approx q_\phi(\nu|\mathbf{Y}) = \prod_{i=1}^N q_\phi(\nu_i|\mathbf{y}_i),$$

where ϕ is a collection of learnable parameters that parametrize the variational distribution and $\mathbf{y}_i$ is all the answer records by student i . Notably, we have removed the dependency of the variational distribution on ψ and θ so that the variational distribution is solely controlled by the variational parameter ϕ . Thus, the design of the variational distribution is highly flexible. All we need to do is to specify a class of distributions and design a function parametrized by ϕ to output the parameters of q_ϕ . Common in prior literature is to use a Gaussian with diagonal covariance for q_ϕ :

$$(3.9) \quad q_\phi(\nu|\mathbf{y}_i) = \mathcal{N}(\mathbf{u}_j, \text{diag}(\mathbf{v}_j)),$$

where its mean and variance $[\mathbf{u}_j^\top, \mathbf{v}_j^\top]^\top = f_\phi(\mathbf{y}_i)$. We can use arbitrarily complex functions such as a deep neural network for f_ϕ as long as they are differentiable; more details in Section 3.3.2. With the above approximation, Bayesian inference turns into an optimization problem under the variational principle, where we now optimize a lower bound, known as the evidence lower bound (ELBO) Bishop [2006], of the marginal data log likelihood. We derive a novel ELBO objective for variational inference applied to FA models in the EDM setting, summarized in the proposition below.

Proposition 1. *In VarFA, the ELBO objective for an FA model in the form of Eq. 3.4 is*

$$(3.10) \quad \mathcal{L}_{\text{ELBO}}(\phi, \theta) = \sum_{i,j \in \Omega_{\text{obs}}} \left(-D_{\text{KL}}[q_{\phi}(\nu_i | \mathbf{y}_i) \| p(\nu_i)] + \mathbb{E}_{\nu_i \sim q_{\phi}(\nu_i | \mathbf{y}_i)} [\log p_{\theta_j}(y_{ij} | \nu_i)] \right)$$

where D_{KL} is the Kullback–Leibler (KL) divergence [MacKay and Mac Kay \[2003\]](#) between two distributions.

PROOF. We start with the marginal data log likelihood which is the objective we want to maximize and introduce the random variables ν_i ’s:

$$\begin{aligned} \log p(\mathbf{Y}) &= \sum_{i,j \in \Omega_{\text{obs}}} \log p_{\theta_j}(y_{ij}) \\ &= \sum_{i,j \in \Omega_{\text{obs}}} \log \int p_{\theta_j}(y_{ij}, \nu_j) d\nu_i \\ &= \sum_{i,j \in \Omega_{\text{obs}}} \log \int q_{\phi}(\nu_i | \mathbf{y}_i) \frac{p_{\theta_j}(y_{ij}, \nu_j)}{q_{\phi}(\nu_i | \mathbf{y}_i)} d\nu_i \\ &\stackrel{(i)}{\geq} \sum_{i,j \in \Omega_{\text{obs}}} \mathbb{E}_{\nu_i \sim q_{\phi}(\nu_i | \mathbf{y}_i)} \left[\log \frac{p_{\theta_j}(y_{ij}, \nu_j)}{q_{\phi}(\nu_i | \mathbf{y}_i)} \right] \\ &\geq \sum_{i,j \in \Omega_{\text{obs}}} -\mathbb{E}_{\nu_i} \left[\log \frac{q_{\phi}(\nu_i | \mathbf{y}_i)}{p(\nu_j)} \right] + \mathbb{E}_{\nu_i} [\log p_{\theta_j}(y_{ij} | \nu_j)] \\ &= \mathcal{L}_{\text{ELBO}}(\phi, \theta) \end{aligned}$$

where step (i) follows from Jensen’s inequality [Jensen et al. \[1906\]](#). □

The ELBO objective differs from those used in existing literature in that, in our case, because the data matrix is only partially observed, the summation is only over

$\{i, j\} \in \Omega_{\text{obs}}$, i.e., the observed entries in the data matrix. In contrast, in prior literature, the summation in the ELBO objective is over all all entries in the data matrix.

We form the following optimization objective to estimate ϕ and θ :

$$(3.11) \quad \hat{\theta}, \hat{\phi} = \underset{\theta, \phi}{\operatorname{argmin}} -\mathcal{L}_{\text{ELBO}}(\phi, \theta) + \lambda \mathcal{R}(\theta),$$

where $\mathcal{R}(\theta)$ is a regularization term. That is, we perform VI on the student factor ν and MLE inference on the remaining factors θ . More explicitly, to perform VI on ν , we simply compute the mean and standard deviation of q_ϕ using f_ϕ with the learnt variational parameters ϕ .

3.3.2. Why is VarFA efficient?

VarFA supports efficient stochastic optimization. By formulating Bayesian inference as an optimization problem via the ELBO objective, VarFA allows efficient optimization algorithms such as SGD, with a few additional tricks. In particular, we require all terms in $\mathcal{L}(\theta, \phi)$ to be differentiable with respect to θ and ϕ which enable gradient computation necessary for SGD. This requirement is easily satisfied with a few moderate assumptions. Specifically, we let the variational distribution q_ϕ and the prior distribution $p(\nu_i)$ for each i to be Gaussian (See Eq. 3.9; we use a standard Gaussian for the prior distribution $p(\nu_i)$, i.e., $p(\nu_i) = \mathcal{N}(0, \mathbf{I})$) following existing literature [Kingma and Welling \[2014\]](#); [Rezende et al. \[2014\]](#). These two assumptions are not particularly limiting especially given the vast number of successful applications of VI that rely on the same assumptions [Zhao et al. \[2019\]](#); [Mathieu et al. \[2019\]](#); [Siddharth et al. \[2017\]](#); [Creager et al. \[2019\]](#); [Chen et al. \[2018\]](#); [Higgins et al. \[2017\]](#); [Kim and Mnih \[2018\]](#). Thanks to

the Gaussian assumption, for the first term in $\mathcal{L}(\theta, \phi)$, we can easily compute the KL divergence term analytically in closed form. For the second term in $\mathcal{L}(\theta, \phi)$, we can use the so-called reparametrization trick, i.e., $\nu_i = \mathbf{u}_i + \mathbf{v}_i \odot \boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, which allows a low variance estimation of the gradient of the second term in $\mathcal{L}(\theta, \phi)$ with respect to ϕ . See Section 2.3 in Kingma and Welling [2014], Section 2.3 in Kingma et al. [2019] and Section 3 in Rezende et al. [2014] for more details on stochastic gradient computation in VI. As a result, our framework can be efficiently implemented using a number of open source, automatic differentiation packages such as Tensorflow Abadi et al. [2015] and PyTorch Paszke et al. [2019].

VarFA supports amortized inference. Classic Bayesian inference methods such as MCMC infer each parameter ν_i for each student. As the number of students increases, the number of parameters that MCMC needs to infer also increases, which may not be scalable in large-scale data settings. In contrast, unlike classic Bayesian inference methods, VI is *amortized*. It does *not* infer each parameter ν_i . Rather, it estimates a *single* set of variational parameter ϕ responsible for inferring *all* ν_i 's. In this way, once we have trained FA models using VarFA and obtain the variational parameter ϕ , we can easily infer ν_i by simply computing q_ϕ , even for new students, without invoking any additional optimization or inference procedures.

3.3.3. Dealing with missing entries

Note from Eq. 3.10 that the variational distribution q_ϕ for each ν_i is conditioned on $\mathbf{y}_i \in \mathbb{R}^Q$, i.e., an entire row in $\mathbf{Y}$. However, in practice, the data matrix $\mathbf{Y}$ is often only partially observed, i.e., some students only answer a subset of questions. Then,

y_i 's will likely contain missing values which computation cannot be performed on. To work around this issue, we use “zero imputation”, a simple strategy that transforms the missing values to 0, following prior work on applying VI in the context of recommender systems [Liang et al. \[2018\]](#); [Salakhutdinov et al. \[2007\]](#); [Sedhain et al. \[2015\]](#); [Nazabal et al. \[2018\]](#) that demonstrated the effectiveness of this strategy despite its simplicity. We note that there exist more elaborate ways to deal with missing entries, such as designing specialized function f_ϕ for the variational distribution [Ma et al. \[2019\]](#); [Gong et al. \[2019\]](#); [Ma et al. \[2018\]](#). We leave the investigation of more effectively dealing with missing entries to future work.

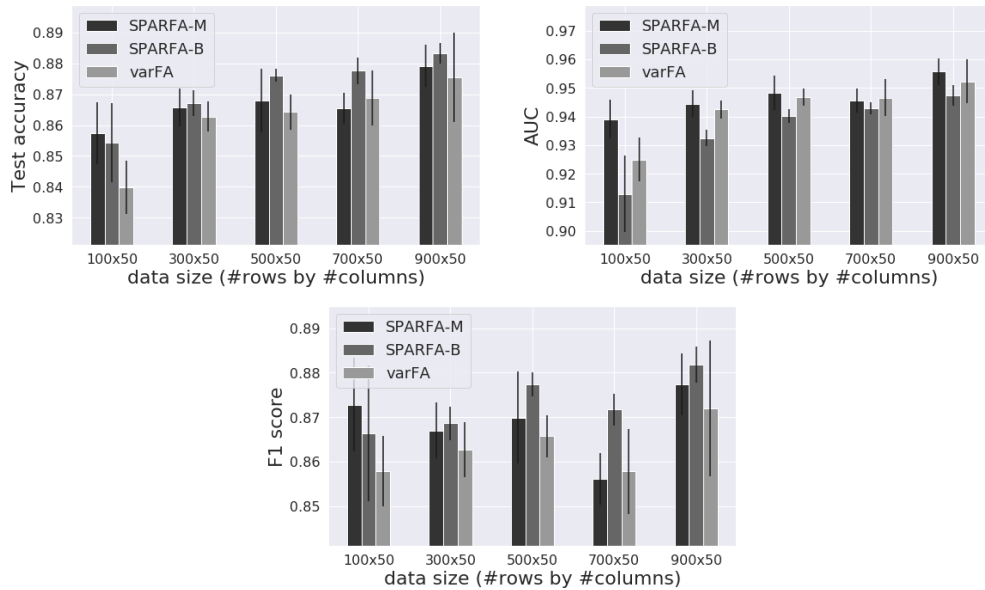


Figure 3.1. Performance of the SPARFA-M, SPARFA-B, and VarFA algorithms on the synthetic data set with different data sizes. Plots from left to right show comparison on accuracy (ACC), area under curve (AUC) and F1 metrics, respectively. Higher is better for all metrics. VarFA performs similarly to SPARFA-M and SPARFA-B.

3.3.4. Remarks

Applicability of VarFA. The VarFA framework is general and flexible and can be applied to a wide array of FA models. The recipe for applying VarFA to an FA model of choice is as follows: 1) formulate the FA model into the canonical formulation as in Eq. 3.4; 2) partition the factors into student factor ν , the remaining unknown factors θ and known factors ψ ; 3) perform VI on ν and MLE estimation on θ , following Section. 3.3.1.

Relation to variational auto-encoders. Our proposed framework can be regarded as a standard variational auto-encoder (VAE) but with the decoder implemented not as a neural network but by the FA model. Because the decoder is constraint to a FA model, it is more interpretable than a neural network.

Relation to other efficient Bayesian factor analysis methods. We acknowledge that VI applied to general FA models have been proposed in existing literature Ghahramani and Beal [2000]; Zhao and Philip [2009]. However, to our knowledge, little prior work have applied VI to educational FA models nor investigated its effectiveness. One concurrent work applied VI to IRT Wu et al. [2020]. Our VarFA framework applies generally to a number of other FA models by following the recipe in the preceding paragraph, including IRT. Thus, our work complements existing literature in providing promising results in applying VI for FA models in the context of educational data mining.

3.4. Experiments

We demonstrate the efficacy of VarFA variational inference framework using the sparse factor analysis model (SPARFA) as the underlying FA model. This choice of SPARFA as the FA model to investigate is motivated by its mathematical generality: it assumes

no auxiliary information is available and all three factors $\mathbf{c}_i$'s, $\mathbf{m}_j$'s and μ_j 's need to be estimated, which makes the inference problem more challenging. Lan et al. [2014] provides two inference algorithms including SPARFA-M (based on MLE) and SPARFA-B (based on MAP). We conduct experiments on both synthetic and real data sets. Using synthetic data sets, we compare VarFA to both SPARFA-M and SPARFA-B. We demonstrate that 1) VarFA predicts students' answers as accurately as SPARFA-M and SPARFA-B; 2) VarFA is almost $100\times$ faster than SPARFA-B. Using real data sets, we compare VarFA to SPARFA-M. We demonstrate that 1) VarFA predicts students' answers more accurately than SPARFA-M; 2) VarFA can output the same insights as SPARFA-M, including point estimate of students' skill levels and questions' associations with skill tags; 3) VarFA can additionally output meaningful uncertainty quantification for student skill levels, which SPARFA-M is incapable of, without sacrifice to computational efficiency. Note that SPARFA-B can also compute uncertainty for small data sets but fails for large data sets due to scalability issues and thus we do not compare to SPARFA-B for real data sets. Specifically for SPARFA, using the notation convention in Section 3.2.2, the question-skill association factors $\mathbf{m}_j$'s and the question difficulty factors μ_j are collected in $\theta = \{\mathbf{m}_1, \dots, \mathbf{m}_Q, \mu_1, \dots, \mu_Q\}$. The student skill level factors $\mathbf{c}_j$'s are collected in $\nu = \{\mathbf{c}_1, \dots, \mathbf{c}_N\}$. Because the factors $\mathbf{m}_j$'s are unknown, the "skills" in SPARFA are *latent* (referred to as "latent skills" in subsequent discussions) and are not attached to any specific interpretation. However, as we will see in Section 3.4.2.4, assuming the skill tags are available as auxiliary information, we can associate the estimated latent skills with the provided skills tags using the same approach proposed in Lan et al. [2014]. For the regularization term in Eq. 3.11, because the factors $\mathbf{m}_j$'s are assumed to be sparse and

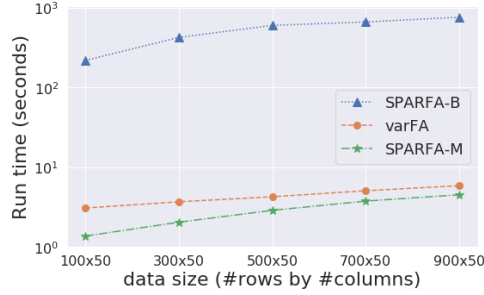


Figure 3.2. Training run time comparison among VarFA, SPARFA-M, and SPARFA-B on synthetic data sets of varying sizes. VarFA performs approximate Bayesian inference almost 100x faster than SPARFA-B and is close to the run time of SPARFA-M. Thus, VarFA enables practical and scalable Bayesian inference for very large data sets.

nonnegative, we use ℓ_1 regularization for $\mathbf{m}_j$'s. When performing MLE for $\mathbf{m}_j$'s, we use the proximal gradient algorithm for optimization problem with nonnegative and sparsity requirements; see Section 3.2 in Lan et al. [2014] for more details. For the other unknown factor μ_j 's in θ , we apply standard ℓ_2 regularization. The code along with instructions to reproduce our experiments can be downloaded from <https://tinyurl.com/tvm4332>.

3.4.1. Synthetic Data Experiments

3.4.1.1. Setup.

Data set generation. We generate data set according to the canonical FA model specified in Eq. 3.4, taking into consideration the additional sparsity and nonnegativity assumptions in SPARFA. Specifically, We sample the factors μ_j 's and $\mathbf{c}_i$'s from i.i.d. standard isotropic Gaussian distributions with variance 1 (or identity matrix for $\mathbf{c}_i$'s) and mean sampled from a uniform distribution in range $[-1, 1]$. To simulate sparse and nonnegative factors $\mathbf{m}_j$'s, we first sample an auxiliary variable $s_{kj} \stackrel{\text{i.i.d.}}{\sim} \text{Bernoulli}(\pi)$, where π controls the sparsity, and then sample $m_{kj} \stackrel{\text{i.i.d.}}{\sim} \text{Exponential}(1)$ when $s_{kj} = 1$ or setting $m_{kj} = 0$

Table 3.1. Summary statistics of pre-processed real data sets.

data set	#students	#questions	%observed
assistent	392	747	12.93%
algebra	697	782	13.02%
bridge	913	1242	11.42%

when $s_{kj} = 0$ for each j and k . In all synthetic data experiments, we use $\pi = 0.3$ and set the true number of latent skills to $K = 5$.

Experimental settings. We vary the size of the data set and use 5 different data matrix sizes: 100×50 , 300×50 , 500×50 , 700×50 and 900×50 . We select a data missing rate of 50%, i.e., we randomly choose 50% of all entries in the data matrix as training set and the rest as test set. Experimental results for each of the above data sizes are averaged over 5 runs where we randomize over the train/test data split. We train SPARFA with both SPARFA-M and VarFA using the Adam optimizer [Kingma and Ba \[2015\]](#) with learning rate = 0.05 for 100 epochs. Regularization hyper-parameters are chosen by grid search for each experiment. For VarFA, we use a simple 3-layer neural network for the function f_ϕ in the variational distribution. We use the hyper-parameter settings for SPARFA-B following Section 4.2 in [Lan et al. \[2014\]](#).

Evaluation metrics. We evaluate the inference algorithms on their ability to recover (predict) the missing entries in the data matrix given the observed entries. We term this criterion “student answer prediction”. Since our data matrix is binary, using prediction accuracy (ACC) alone does not accurately reflect the performance of the inference algorithms under comparison. Thus, we use area under the receiver operating characteristic curve (AUC) and F1 score in addition to ACC, as standard in evaluating binary predictions [Hastie et al. \[2009\]](#).

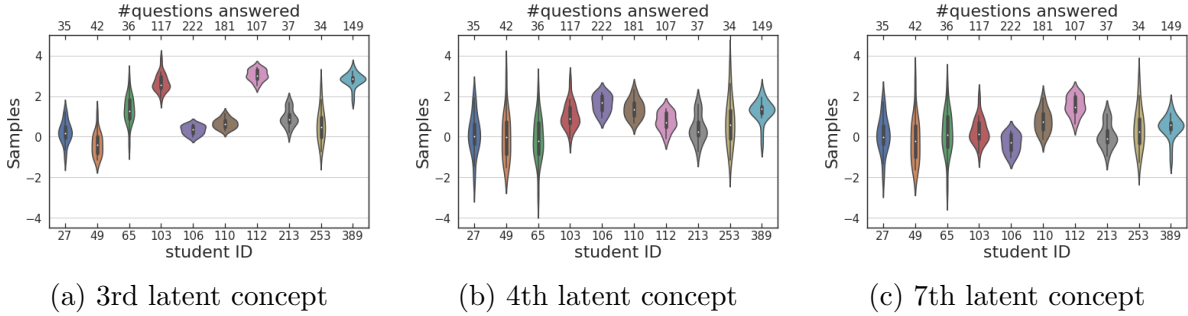


Figure 3.3. Violin plot showing the mean and standard deviation of the estimated skill mastery levels on 10 selected students on the 3rd, 4th and 7th latent skills that VarFA computes. In each sub-figure, bottom and top axes respectively shows student IDs and top axis shows the number of questions each student answered. The more questions a student answers, the tighter the credible interval. (Best viewed in color.)

3.4.1.2. Results. Fig. 3.1 shows bar plots that compare the student answer prediction performance of VarFA with SPARFA-M and SPARFA-B on all three evaluation metrics. We can observe that all three methods perform similarly and that SPARFA-M and SPARFA-B show no statistically significant advantage over VarFA. We further showcase VarFA’s scalability by comparing its training run-time with SPARFA-M and SPARFA-B. The results are shown in Fig. 3.2. VarFA is significantly faster than SPARFA-B and is almost as fast as SPARFA-M.

In summary, with VarFA, we obtain posteriors that allow uncertainty quantification with roughly the *same* computation complexity of computing point estimates and *without* compromising prediction performance.

3.4.2. Real Data Experiments

3.4.2.1. Setup.

Table 3.2. Student answer prediction performance comparing VarFA to SPARFA-M on Assistment, Algebra and Bridge data sets. $\uparrow$ and $\downarrow$ denote higher and lower is better, respectively. VarFA performs better than SPARFA-M on all three data sets and evaluation metrics most of the time. Additionally, VarFA’s run time is very close to SPARFA-M.

(a) Assistment		
Metric	Algorithm	
	SPARFA-M	VarFA
ACC $\uparrow$	0.7074 $\pm$ 0.0044	0.7101 $\pm$ 0.0048
AUC $\uparrow$	0.756 $\pm$ 0.048	0.7635 $\pm$ 0.0036
F1 $\uparrow$	0.7746 $\pm$ 0.0029	0.7765 $\pm$ 0.0014
Run time (s) $\downarrow$	5.3319 $\pm$ 0.2774	6.9167 $\pm$ 0.1074
(b) Algebra		
Metric	Algorithm	
	SPARFA-M	VarFA
ACC $\uparrow$	0.7735 $\pm$ 0.0037	0.7774 $\pm$ 0.0031
AUC $\uparrow$	0.8137 $\pm$ 0.003	0.8245 $\pm$ 0.002
F1 $\uparrow$	0.8465 $\pm$ 0.0021	0.8486 $\pm$ 0.001
Run time (s) $\downarrow$	8.464 $\pm$ 0.4568	10.3335 $\pm$ 0.4435
(c) Bridge		
Metric	Algorithm	
	SPARFA-M	VarFA
ACC $\uparrow$	0.8492 $\pm$ 0.0016	0.8468 $\pm$ 0.0016
AUC $\uparrow$	0.837 $\pm$ 0.0024	0.8419 $\pm$ 0.0028
F1 $\uparrow$	0.9121 $\pm$ 0.0005	0.912 $\pm$ 0.0009
Run time (s) $\downarrow$	15.6048 $\pm$ 0.7314	15.8558 $\pm$ 1.046

Data sets and pre-processing steps. We perform experiments on three large-scale, publicly available, real educational data sets including ASSISTments 2009-2010 (Assistment) [Heffernan and Heffernan \[2014\]](#), Algebra I 2006-2007 (algebra) [Stamper et al. \[2010\]](#) and Bridge to Algebra 2006-2007 (bridge) [Stamper et al. \[2010\]](#); [Stamper and Pardos](#)

[2016]. The details of the data sets, including data format and data collection procedure can be found in the preceding references. We remove students and questions that have too few answer records from the data sets to reduce the sparsity of the data sets. Specifically, we keep students and questions that have no less than 30, 35 and 40 answer records for Assistment, Algebra and Bridge data sets, respectively. In each data set, each student may provide more than one answer record for each question. Therefore, we also remove student-question answer records except for the first one. Table 3.1 presents the summary statistics of the resulting pre-processed data matrix for each data set.

Experimental settings and evaluation metrics. Optimizer, learning rate, number of epochs, neural network architecture and evaluation metrics are the same as in synthetic data experiments. For real data sets, we use 8 latent skills instead of 5. Other choices of the number of latent skills might result in better performance. However, since we are comparing different inference algorithms for the *same* model, it is a fair to compare the inference algorithms on the same model with the same number of latent dimensions. We use a 80:20 data split, i.e., we randomly sample 80% and 20% of the observed entries in each data set, without replacement, as training and test sets, respectively. Regularization hyper-parameters are selected by grid search and are different for each data set. We only compare VarFA with SPARFA-M because SPARFA-B does not scale to such large data sets.

3.4.2.2. Results: Performance Comparison. Table 3.2 shows the average performance on the test set of each data set comparing VarFA and SPARFA-M for all three data sets and additionally run time. We can see that VarFA achieves slightly better student answer prediction on most data sets and on most metrics. Interestingly, recall

that, in the preceding synthetic data set experiment, SPARFA-M performed better than VarFA most of the time. A possible explanation is that, for synthetic data sets, the underlying data generation process matches the SPARFA model, whereas for real data sets, the underlying data generation process is unknown. VarFA’s better performance than SPARFA-M for real data sets implies that VarFA is more robust to the unknown underlying data generation process. As a result, VarFA may be more applicable than SPARFA-M in real-world situations.

Table 3.2 also shows the run time comparison between VarFA and SPARFA-M; see the last row in each sub-table. We see that both inference algorithms have very similar run time, showing that VarFA is applicable for very large data sets. Notably, VarFA achieves this efficiency while also performing Bayesian inference on the student knowledge level factor.

3.4.2.3. Results: Bayesian Inference With VarFA. We now illustrate VarFA’s capability of outputting credible intervals using the Assistment data set. Fig. 3.3 presents violin plots that show the sampled student latent skill levels for a random subset of 10 students. Plots 3.3a, 3.3b and 3.3c shows the inferred students ability for the 3rd, 4th and 7th latent skill dimension. In each plot, the bottom axis shows the student ID and the top axis shows the total number of questions answered by the corresponding student. For each student, the horizontal width of the violin represents the density of the samples; the skinnier the violin, the more widespread the samples are, implying the model’s less certainty on its estimations.

Results in Fig. 3.3 confirms our intuition that the more questions a student answers, the more certain the model is about its estimation. For example, students with ID 106,

Table 3.3. Illustration of the estimated latent skills with the their top 3 most strongly associated skill tags in the Assisment data set. The percentage in the parenthesis shows the association probability (summed to 1 for each latent skill). We see that the tagged skills associated with each estimated latent skill form intuitive and interpretable groups.

Latent Skill 1	Latent Skill 3
Division Fractions (29.1%)	Conversion of Fraction Decimals Percents (7.3%)
Least Common Multiple (18.1%)	Addition and Subtraction Positive Decimals (6.8%)
Write Linear Equation from Ordered Pairs (17.8%)	Probability of a Single Event (5.7%)
Latent Skill 4	Latent Skill 7
Pattern Finding (17.4%)	Volume Sphere (13.4%)
Histogram as Table or Graph (11.3%)	Volume Cylinder (10.4%)
Percent Of (10.5%)	Surface Area Rectangular Prism (10.2%)

110 and 389 answered 222, 181 and 149 questions, respectively, and the credible intervals of their ability estimation is quite small. In contrast, students with ID 27, 49 and 65 answered far less questions and the credible intervals of their ability estimation is quite large. This result implies that VarFA outputs sensible and interpretable credible intervals. As mentioned in Section 3.1, such uncertainty quantification may benefit a number of educational applications such as improving adaptive testing algorithms. Note that VarFA is able to compute such credible interval as fast as SPARFA-M, making VarFA potentially useful for even real-time educational systems. In contrast, SPARFA-M is not capable of computing credible intervals. Although we may still obtain confidence interval as uncertainty quantification with SPARFA-M via bootstrapping, i.e., train with SPARFA-M multiple times with random subsets of the data set and then use the point estimates from different random runs to compute confidence intervals. However, this method suffers from two immediate drawbacks: 1) it is slow because we need to run the model multiple times and 2) different runs result in permutation in the estimated factors and thus the averaged estimations loss meaning. Given that VarFA is as efficient as SPARFA-M and

the previous drawbacks of SPARFA-M, we recommend VarFA over bootstrapping with SPARFA-M for quantifying model’s uncertainty in practice.

3.4.2.4. Results: Post-Processing for Improved Interpretability. SPARFA assumes that each student factor ν_i identifies a multi-dimensional skill level on a number of “latent” skills (recall that we use 8 latent skills in our experiments). As mentioned earlier, these latent skills are not interpretable without the aid of additional information. To improve interpretability, [Lan et al. \[2014\]](#) proposed that, when the skill tags for each question is available in the data set, we can associate each latent skill with skill tags via a simple matrix factorization. Then, we can compute each students’ mastery levels on the actual skill tags. We refer readers to Sections 5.1 and 5.2 in [Lan et al. \[2014\]](#) for more technical details.

Although the above method to improve interpretability has already been proposed, [Lan et al. \[2014\]](#) only presented results on private data sets, whereas here we presents results on publicly available data set with code, making our results more transparent and reproducible.

We again use the Assistment data set for illustration. We compute the association of skill tags in the data set with each of the latent skills and show 4 of the latent skills with their top 3 most strongly associated skill tags. We can see that each latent skill roughly identify the same group of skill tags. For example, latent skill 4 clusters skill tags on statistics and probability while latent skill 7 clusters skill tags on geometry. Thus, by simple post-processing, we obtain an interpretation of the latent skills by associating them with known skill tags in the data.

We can similarly obtain VarFA’s estimations of the students’ mastery levels on each skill tags through the above process. In Fig. 3.4, we compare the predicted mastery level for each skill tag (only for the questions this student answered) with the percent of correct answers for that skill tag. Blue curve shows the empirical student’s mastery level on a skill tag by computing the percentage of correctly answered questions belonging to a particular skill tag. Orange curve shows VarFA’s estimated student mastery level on a skill tag, normalized to range $[0, 1]$. We can see that, when the student gave more correct answers to questions of a particular skill tag, such as skill tag ID 2, 10, 14 and 30, VarFA also predicts a higher ability score for these skill tags. When the student gave more incorrect answers to questions of a particular skill tag, such as skill tag ID 0, 4 and 27, VarFA also predicts a lower ability score for those skill tags. Although the correspondence is not perfect, VarFA’s predicted student’s mastery levels match our intuition about student’s abilities (using the observed correct answer ratio as an empirical estimate) reasonably well. Thus, by post-processing, we can interpret VarFA’s estimated students’ latent skill mastery levels using the easily understandable skill tags.

3.5. Chapter Conclusion and Future Work

In this chapter, we have presented VarFA, a variational inference factor analysis framework to perform efficient Bayesian inference for learning analytics. VarFA is general and can be applied to a wide array of FA models. We have demonstrated the effectiveness of our VarFA using the sparse factor analysis (SPARFA) model as a case study. We have shown that VarFA can very efficiently output interpretable, educationally meaningful information, in particular credible intervals, much faster than classic Bayesian inference

methods. Thus, VarFA has potential application in many educational data mining scenarios where efficient credible interval computation is desired, i.e., in adaptive testing and adaptive learning systems. We have also provided open-source code to reproduce our results and facilitate further research efforts.

We outline three possible future research directions and extensions. First, VarFA currently performs Bayesian inference on the student skill mastery level factor. We are working on extending the framework to perform full Bayesian inference for all unknown factors. Extensions to some models such as IRT is straightforward, as studied in [Wu et al. \[2020\]](#), because all factors can be reasonably assumed to be Gaussian.

However, some other models involve additional modeling assumptions, making it challenging to design distributions that satisfy these assumptions. For example, in SPARFA, one of the factors is assumed to be nonnegative and sparse [Lan et al. \[2014\]](#). No standard distribution fulfill these requirements. We are exploring *mixture* distributions, in particular spike and slab models [Ishwaran et al. \[2005\]](#) for Bayesian variable selection, and methods to combine them with variational inference, following [Titsias and Lázaro-Gredilla \[2011\]](#); [Tonolini et al. \[2019\]](#).

Second, other methods exist that accelerate classic Bayesian inference. Recent works in large-scale Bayesian inference proposed *approximate* MCMC methods that scale to large data sets [Zhang et al. \[2020\]](#); [Ma et al. \[2015\]](#); [Song et al. \[2017\]](#). Some of these methods apply variational inference to perform the approximation [De Freitas et al. \[2001\]](#); [Habib and Barber \[2019\]](#). We are investigating extending VarFA to support approximate MCMC methods.

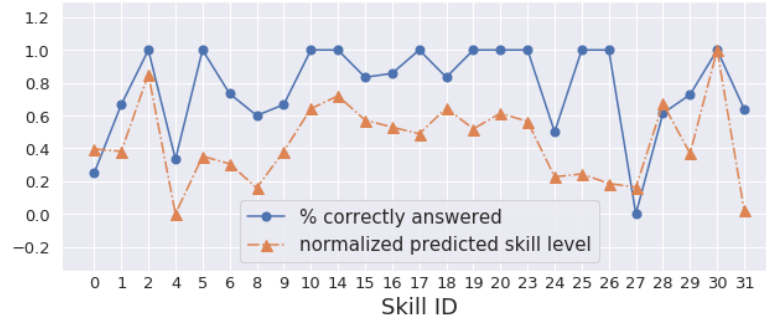


Figure 3.4. Comparison between the estimated skill mastery levels using VarFA’s predictions and using empirical observations for student with ID 110. Even though the two curves show different numeric values, they nevertheless demonstrate similar trends, showing that the predictions reasonably match our intuition about student’s skill mastery levels.

Finally, the goal of VarFA is to improve real-world educational systems and ultimately improve learning. Therefore, it is necessary to evaluate VarFA beyond synthetic and benchmark data sets. We plan to integrate VarFA into an existing educational system and conduct a case study where students interact with the system in real-time to understand the VarFA’s educational implications in the wild.

Bibliography

- [1] Robin A. Moser and Gábor Tardos. A constructive proof of the general lovász local lemma. *J. ACM*, 57(2):11:1–11:15, 2010.
- [2] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Comput.*, 14(8):1771–1800, 2002.
- [3] Ioannis Tsochantaridis, Thorsten Joachims, Thomas Hofmann, and Yasemin Altun. Large margin methods for structured and interdependent output variables. *J. Mach. Learn. Res.*, 6:1453–1484, December 2005.
- [4] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C. Courville, and Yoshua Bengio. Generative adversarial nets. In *NIPS*, pages 2672–2680, 2014.
- [5] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *ICLR*, 2014.
- [6] Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. MADE: masked autoencoder for distribution estimation. In *ICML*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 881–889. JMLR.org, 2015.
- [7] Hugo Larochelle and Iain Murray. The neural autoregressive distribution estimator. In *AISTATS*, volume 15 of *JMLR Proceedings*, pages 29–37. JMLR.org, 2011.
- [8] Aäron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *ICML*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1747–1756. JMLR.org, 2016.
- [9] Martín Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *ICML*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223. PMLR, 2017.
- [10] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *NeurIPS*, pages 11895–11907, 2019.
- [11] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*. OpenReview.net, 2021.
- [12] Kevin P. Murphy, Yair Weiss, and Michael I. Jordan. Loopy belief propagation for approximate inference: An empirical study. In *UAI*, pages 467–475. Morgan Kaufmann, 1999.
- [13] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. Generalized belief propagation. In *NIPS*, pages 689–695. MIT Press, 2000.

- [14] Martin J. Wainwright and Michael I. Jordan. Log-determinant relaxation for approximate inference in discrete markov random fields. *IEEE Trans. Signal Process.*, 54(6-1):2099–2109, 2006.
- [15] A. Braunstein, M. Mézard, and R. Zecchina. Survey propagation: an algorithm for satisfiability. *Random Struct. Algorithms*, 27:201–226, 2005.
- [16] Henrik Reif Andersen, Tarik Hadzic, John N. Hooker, and Peter Tiedemann. A constraint store based on multivalued decision diagrams. In *CP*, volume 4741 of *Lecture Notes in Computer Science*, pages 118–132. Springer, 2007.
- [17] Mark Chavira, Adnan Darwiche, and Manfred Jaeger. Compiling relational bayesian networks for exact inference. *Int. J. Approx. Reason.*, 42(1-2):4–20, 2006.
- [18] Pascal Van Hentenryck. *Constraint Satisfaction in Logic Programming*. MIT Press, Cambridge, MA, USA, 1989. ISBN 0-262-08181-4.
- [19] Vibhav Gogate and Rina Dechter. Importance sampling-based estimation over and/or search spaces for graphical models. *Artificial Intelligence*, 184-185:38 – 77, 2012.
- [20] Tian Sang, Paul Bearne, and Henry Kautz. Performing bayesian inference by weighted model counting. In *AAAI*, AAAI’05, pages 475–481, 2005.
- [21] Matt J. Kusner, Brooks Paige, and José Miguel Hernández-Lobato. Grammar variational autoencoder. In *ICML*, volume 70, pages 1945–1954. PMLR, 2017.
- [22] Wengong Jin, Regina Barzilay, and Tommi S. Jaakkola. Junction tree variational autoencoder for molecular graph generation. In *ICML*, volume 80 of *Proceedings of Machine Learning Research*, pages 2328–2337. PMLR, 2018.
- [23] Hanjun Dai, Yingtao Tian, Bo Dai, Steven Skiena, and Le Song. Syntax-directed variational autoencoder for structured data. In *ICLR (Poster)*. OpenReview.net, 2018.
- [24] Zhiting Hu, Zichao Yang, Xiaodan Liang, Ruslan Salakhutdinov, and Eric P. Xing. Toward controlled generation of text. In *ICML*, volume 70 of *Proceedings of Machine Learning Research*, pages 1587–1596. PMLR, 2017.
- [25] Daniel Lowd and Pedro M. Domingos. Learning arithmetic circuits. In *UAI*, pages 383–392. AUAI Press, 2008.
- [26] Fan Ding, Jianzhu Ma, Jinbo Xu, and Yexiang Xue. XOR-CD: linearly convergent constrained structure generation. In *ICML*, volume 139 of *Proceedings of Machine Learning Research*, pages 2728–2738. PMLR, 2021.
- [27] Paul Erdős and László Lovász. Problems and results on 3-chromatic hypergraphs and some related questions. In *Colloquia Mathematica Societatis Janos Bolyai 10. Infinite and Finite Sets, Keszthely (Hungary)*. Citeseer, 1973.
- [28] Heng Guo, Mark Jerrum, and Jingcheng Liu. Uniform sampling through the lovász local lemma. *J. ACM*, 66(3):18:1–18:31, 2019.
- [29] Daniel Selsam, Matthew Lamm, Benedikt Bünz, Percy Liang, Leonardo de Moura, and David L. Dill. Learning a SAT solver from single-bit supervision. In *ICLR (Poster)*. OpenReview.net, 2019.

- [30] Haonan Duan, Pashootan Vaezipoor, Max B. Paulus, Yangjun Ruan, and Chris J. Maddison. Augment with care: Contrastive learning for combinatorial problems. In *ICML*, volume 162 of *Proceedings of Machine Learning Research*, pages 5627–5642. PMLR, 2022.
- [31] Zhuwen Li, Qifeng Chen, and Vladlen Koltun. Combinatorial optimization with graph convolutional networks and guided tree search. In *NeurIPS*, pages 537–546, 2018.
- [32] Emre Yolcu and Barnabás Póczos. Learning local search heuristics for boolean satisfiability. In *NeurIPS*, pages 7990–8001, 2019.
- [33] Xinyun Chen and Yuandong Tian. Learning to perform local rewriting for combinatorial optimization. In *NeurIPS*, pages 6278–6289, 2019.
- [34] Nikolaos Karalias and Andreas Loukas. Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. In *NeurIPS*, pages 6659–6672, 2020.
- [35] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *Eur. J. Oper. Res.*, 290(2): 405–421, 2021.
- [36] Jayanta Mandi, Emir Demirovic, Peter J. Stuckey, and Tias Guns. Smart predict-and-optimize for hard combinatorial optimization problems. In *AAAI*, pages 1603–1610. AAAI Press, 2020.
- [37] Radford M Neal. *Probabilistic inference using Markov chain Monte Carlo methods*. Department of Computer Science, University of Toronto Toronto, ON, Canada, 1993.
- [38] Paul Dagum and R. Martin Chavez. Approximating probabilistic inference in bayesian belief networks. *IEEE Trans. Pattern Anal. Mach. Intell.*, 15(3):246–255, 1993.
- [39] Hong Ge, Kai Xu, and Zoubin Ghahramani. Turing: A language for flexible probabilistic inference. In *AISTATS*, volume 84, pages 1682–1690. PMLR, 09–11 Apr 2018.
- [40] Alexander T. Ihler, John W. Fisher III, and Alan S. Willsky. Loopy belief propagation: Convergence and effects of message errors. *J. Mach. Learn. Res.*, 6:905–936, 2005.
- [41] Amin Coja-Oghlan, Noëla Müller, and Jean B. Ravelomanana. Belief propagation on the random k-sat model, 2020.
- [42] Fan Ding and Yexiang Xue. Contrastive divergence learning with chained belief propagation. In *PGM*, volume 138 of *Proceedings of Machine Learning Research*, pages 161–172. PMLR, 2020.
- [43] Vibhav Gogate and Rina Dechter. Samplesearch: Importance sampling in presence of determinism. *Artif. Intell.*, 175(2):694–729, 2011.
- [44] Carla P. Gomes, Ashish Sabharwal, and Bart Selman. Near-uniform sampling of combinatorial spaces using XOR constraints. In *NIPS*, pages 481–488. MIT Press,

- 2006.
- [45] Stefano Ermon, Carla P. Gomes, Ashish Sabharwal, and Bart Selman. Taming the curse of dimensionality: Discrete integration by hashing and optimization. In *ICML (2)*, volume 28 of *JMLR Workshop and Conference Proceedings*, pages 334–342. JMLR.org, 2013.
 - [46] Dimitris Achlioptas and Panos Theodoropoulos. Probabilistic model counting with short xors. In *SAT*, volume 10491 of *Lecture Notes in Computer Science*, pages 3–19. Springer, 2017.
 - [47] Supratik Chakraborty, Kuldeep S. Meel, and Moshe Y. Vardi. A scalable and nearly uniform generator of SAT witnesses. In *CAV*, volume 8044, pages 608–623. Springer, 2013.
 - [48] Yishay Mansour. Learning boolean functions via the fourier transform. In *Theoretical advances in neural computation and learning*, pages 391–424. Springer, 1994.
 - [49] Chris K Carter and Robert Kohn. On gibbs sampling for state space models. *Biometrika*, 81(3):541–553, 1994.
 - [50] Rahul Gupta, Shubham Sharma, Subhajit Roy, and Kuldeep S. Meel. WAPS: weighted and projected sampling. In *TACAS*, volume 11427, pages 59–76, 2019.
 - [51] Supratik Chakraborty, Daniel J. Fremont, Kuldeep S. Meel, Sanjit A. Seshia, and Moshe Y. Vardi. Distribution-aware sampling and weighted model counting for SAT. In *AAAI*, pages 1722–1730. AAAI Press, 2014.
 - [52] Stefano Ermon, Carla P. Gomes, Ashish Sabharwal, and Bart Selman. Embed and project: Discrete sampling with universal hashing. In *NIPS*, pages 2085–2093, 2013.
 - [53] Fan Ding and Yexiang Xue. XOR-SGD: provable convex stochastic optimization for decision-making under uncertainty. In *UAI*, volume 161 of *Proceedings of Machine Learning Research*, pages 151–160. AUAI Press, 2021.
 - [54] Priyanka Golia, Mate Soos, Sourav Chakraborty, and Kuldeep S. Meel. Designing samplers is easy: The boon of testers. In *FMCAD*, pages 222–230. IEEE, 2021.
 - [55] Rafael Dutra, Kevin Laeufer, Jonathan Bachrach, and Koushik Sen. Efficient sampling of SAT solutions for testing. In *ICSE*, pages 549–559. ACM, 2018.
 - [56] Mate Soos, Stephan Gocht, and Kuldeep S. Meel. Tinted, detached, and lazy CNF-XOR solving and its applications to counting and sampling. In *CAV (1)*, volume 12224 of *Lecture Notes in Computer Science*, pages 463–484. Springer, 2020.
 - [57] Shubham Sharma, Rahul Gupta, Subhajit Roy, and Kuldeep S. Meel. Knowledge compilation meets uniform sampling. In *LPAR*, volume 57 of *EPiC Series in Computing*, pages 620–636, 2018.
 - [58] Carmine Dodaro and Alessandro Previti. Minipref: A tool for preferences in SAT (short paper). In *RCRA + RiCeRcA*, volume 2538 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2019.
 - [59] Massimo Lauria, Jan Elffers, Jakob Nordström, and Marc Vinyals. Cnfgen: A generator of crafted benchmarks. In *SAT*, volume 10491, pages 464–473. Springer, 2017.

- [60] Alexey Ignatiev, António Morgado, and João Marques-Silva. Pysat: A python toolkit for prototyping with SAT oracles. In *SAT*, volume 10929 of *Lecture Notes in Computer Science*, pages 428–437. Springer, 2018.
- [61] Sourav Chakraborty and Kuldeep S. Meel. On testing of uniform samplers. In *AAAI*, pages 7777–7784, 2019.
- [62] Henry Cohn, Robin Pemantle, and James Gary Propp. Generating a random sink-free orientation in quadratic time. *Electron. J. Comb.*, 9(1), 2002.
- [63] Junji Takahashi, Takuya Yamaguchi, Kosuke Sekiyama, and Toshio Fukuda. Communication timing control and topology reconfiguration of a sink-free meshed sensor network with mobile robots. *IEEE/ASME transactions on mechatronics*, 14(2):187–197, 2009.
- [64] Mark Jerrum. Fundamentals of partial rejection sampling, 2021.
- [65] Emanuele Di Rosa, Enrico Giunchiglia, and Barry O’Sullivan. Optimal stopping methods for finding high quality solutions to satisfiability problems with preferences. In *SAC*, pages 901–906. ACM, 2011.
- [66] Nicolas Prevot, Mate Soos, and Kuldeep S. Meel. Leveraging gpus for effective clause sharing in parallel SAT solving. In *SAT*, pages 471–487, 2021.
- [67] Muhammad Osama Mahmoud. *GPU Enabled Automated Reasoning*. PhD thesis, Mathematics and Computer Science, March 2022.
- [68] Johannes Klaus Fichte, Markus Hecher, and Markus Zisser. An improved gpu-based SAT model counter. In *CP*, pages 491–509, 2019.
- [69] Kurt VanLehn. Student modeling. *Foundations of Intelligent Tutoring Systems*, 55: 78, 1988.
- [70] Konstantina Chrysafiadi and Maria Virvou. Student modeling approaches: A literature review for the last decade. *Expert Systems with Applications*, 40(11):4715–4729, 2013.
- [71] Valerie J Shute and Joseph Psotka. Intelligent tutoring systems: Past, present, and future., 1994.
- [72] Sabine Graf and Kinshuk. *Personalized Learning Systems*, pages 2594–2596. Springer US, Boston, MA, 2012. ISBN 978-1-4419-1428-6. URL https://doi.org/10.1007/978-1-4419-1428-6_152.
- [73] John R Anderson, Albert T Corbett, Kenneth R Koedinger, and Ray Pelletier. Cognitive tutors: Lessons learned. *Journal of the Learning Sciences*, 4(2):167–207, 1995.
- [74] Benjamin D Nye, Arthur C Graesser, and Xiangen Hu. Autotutor and family: A review of 17 years of natural language tutoring. *International Journal of Artificial Intelligence in Education*, 24(4):427–469, 2014.
- [75] Neil T Heffernan and Cristina Lindquist Heffernan. The assistments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*, 24(4):470–497, 2014.

- [76] Anna N Rafferty, Michelle M LaMar, and Thomas L Griffiths. Inferring learners' knowledge from their actions. *Cognitive Science*, 39(3):584–618, 2015.
- [77] Cristóbal Romero and Sebastián Ventura. Educational data mining: a review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics*, 40(6):601–618, 2010.
- [78] Ryan SJD Baker and Kalina Yacef. The state of educational data mining in 2009: A review and future visions. *Journal of Educational Data Mining*, 1(1):3–17, 2009.
- [79] Agathe Merceron and Kalina Yacef. Educational data mining: a case study. In *Proc. Artificial Intelligence in Education*, pages 467–474, 2005.
- [80] Wim J van der Linden and Ronald K Hambleton. *Handbook of modern item response theory*. Springer Science & Business Media, 2013.
- [81] Terry A Ackerman. Using multidimensional item response theory to understand what items and tests are measuring. *Applied Measurement in Education*, 7(4):255–278, 1994.
- [82] Hao Cen, Kenneth Koedinger, and Brian Junker. Learning factors analysis – a general method for cognitive model evaluation and improvement. In Mitsuru Ikeda, Kevin D. Ashley, and Tak-Wai Chan, editors, *Proc. Intelligent Tutoring Systems*, pages 164–175, 2006. ISBN 978-3-540-35160-3.
- [83] Philip I. Pavlik, Hao Cen, and Kenneth R. Koedinger. Performance factors analysis – a new alternative to knowledge tracing. In *Proc. Artificial Intelligence in Education*, page 531–538, NLD, 2009. IOS Press. ISBN 9781607500285.
- [84] Robert V Lindsey, Jeffery D Shroyer, Harold Pashler, and Michael C Mozer. Improving students' long-term knowledge retention through personalized review. *Psychological Science*, 25(3):639–647, 2014.
- [85] Michael C Mozer and Robert V Lindsey. Predicting and improving memory retention: Psychological theory matters in the big data era. In *Big Data in Cognitive Science*, pages 43–73. Psychology Press, 2016.
- [86] Benoît Choffin, Fabrice Popineau, Yolaine Bourda, and Jill-Jënn Vie. Das3h: Modeling student learning and forgetting for optimally scheduling distributed practice of skills. In *Proc. Educational Data Mining*, 2019.
- [87] Jill-Jënn Vie and Hisashi Kashima. Knowledge tracing machines: Factorization machines for knowledge tracing. In *Proc. AAAI Conference on Artificial Intelligence*, volume 33, pages 750–757, 2019.
- [88] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Proc. Advances in Neural Information Processing Systems*, pages 505–513. Curran Associates, Inc., 2015.
- [89] Jiani Zhang, Xingjian Shi, Irwin King, and Dit-Yan Yeung. Dynamic key-value memory networks for knowledge tracing. In *Proc. International Conference on World Wide Web*, pages 765–774, 2017.

- [90] Sein Minn, Yi Yu, Michel C Desmarais, Feida Zhu, and Jill-Jênn Vie. Deep knowledge tracing and dynamic student classification for knowledge tracing. In *Proc. IEEE International Conference on Data Mining*, pages 1182–1187. IEEE, 2018.
- [91] Kevin H Wilson, Yan Karklin, Bojian Han, and Chaitanya Ekanadham. Back to the basics: Bayesian extensions of irt outperform neural networks for proficiency estimation. In *Proc. Educational Data Mining*, 2016.
- [92] Hua-Hua Chang, Zhiliang Ying, et al. Nonlinear sequential designs for logistic item response theory models with applications to computerized adaptive tests. *The Annals of Statistics*, 37(3):1466–1488, 2009.
- [93] Duanli Yan, Alina A Von Davier, and Charles Lewis. *Computerized multistage testing: Theory and applications*. CRC Press, 2016.
- [94] Andrew Gelman, John B Carlin, Hal S Stern, David B Dunson, Aki Vehtari, and Donald B Rubin. *Bayesian data analysis*. Chapman and Hall/CRC, 2013.
- [95] Walter R Gilks, Sylvia Richardson, and David Spiegelhalter. *Markov chain Monte Carlo in practice*. Chapman and Hall/CRC, 1995.
- [96] Andrew S. Lan, Andrew E. Waters, Christoph Studer, and Richard G. Baraniuk. Sparse factor analysis for learning and content analytics. *Journal of Machine Learning Research*, 15:1959–2008, 2014.
- [97] Jean-Paul Fox. *Bayesian item response modeling: Theory and applications*. Springer Science & Business Media, 2010.
- [98] Zachary A Pardos, Neil T Heffernan, Brigham Anderson, Cristina L Heffernan, and Worcester Public Schools. Using fine-grained skill models to fit student performance with bayesian networks. *Handbook of Educational Data Mining*, 417, 2010.
- [99] Edgar C Merkle. A comparison of imputation methods for bayesian factor analysis models. *Journal of Educational and Behavioral Statistics*, 36(2):257–276, 2011.
- [100] Marcy Perkins Driscoll. *Psychology of learning for instruction*. Allyn & Bacon, 1994.
- [101] Kikumi K. Tatsuoka. Rule space: An approach for dealing with misconceptions based on item response theory. *Journal of Educational Measurement*, 20(4):345–354, 1983. ISSN 00220655, 17453984. URL <http://www.jstor.org/stable/1434951>.
- [102] Min Chi, Kenneth R Koedinger, Geoffrey J Gordon, Pamela Jordon, and Kurt VanLahn. Instructional factors analysis: A cognitive model for multiple instructional interventions. *Educational Data Mining*, 2011.
- [103] Jean-Paul Fox and Cees AW Glas. Bayesian estimation of a multilevel irt model using gibbs sampling. *Psychometrika*, 66(2):271–288, 2001.
- [104] Robert J Mislevy. Bayes modal estimation in item response models. *Psychometrika*, 51(2):177–195, 1986.
- [105] Christopher M Bishop. *Pattern recognition and machine learning*. springer, 2006.
- [106] David JC MacKay and David JC Mac Kay. *Information theory, inference and learning algorithms*. Cambridge University Press, 2003.

- [107] Johan Ludvig William Valdemar Jensen et al. Sur les fonctions convexes et les inégalités entre les valeurs moyennes. *Acta Mathematica*, 30:175–193, 1906.
- [108] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic back-propagation and approximate inference in deep generative models. In *Proc. International Conference on Machine Learning*, ICML’14, page II–1278–II–1286. JMLR.org, 2014.
- [109] Shengjia Zhao, Jiaming Song, and Stefano Ermon. Infovae: Balancing learning and inference in variational autoencoders. In *Proc. AAAI Conference on Artificial Intelligence*, volume 33, pages 5885–5892, Feb. 2019.
- [110] Emile Mathieu, Tom Rainforth, N Siddharth, and Yee Whye Teh. Disentangling disentanglement in variational autoencoders. In *Proc. International Conference on Machine Learning*, pages 4402–4412, Jun 2019.
- [111] Narayanaswamy Siddharth, Brooks Paige, Jan-Willem Van de Meent, Alban Desmaison, Noah Goodman, Pushmeet Kohli, Frank Wood, and Philip Torr. Learning disentangled representations with semi-supervised deep generative models. In *Proc. Advances in Neural Information Processing Systems*, pages 5925–5935, Dec. 2017.
- [112] Elliot Creager, David Madras, Joern-Henrik Jacobsen, Marissa Weis, Kevin Swersky, Toniann Pitassi, and Richard Zemel. Flexibly fair representation learning by disentanglement. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proc. Machine Learning Research*, pages 1436–1445, Long Beach, California, USA, 09–15 Jun 2019. PMLR.
- [113] Tian Qi Chen, Xuechen Li, Roger B Grosse, and David K Duvenaud. Isolating sources of disentanglement in variational autoencoders. In *Proc. Advances in Neural Information Processing Systems*, pages 2610–2620, 2018.
- [114] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. In *Proc. International Conference on Learning Representations*, 2017.
- [115] Hyunjik Kim and Andriy Mnih. Disentangling by factorising. In *Proc. International Conference on Machine Learning*, volume 80, pages 2649–2658, 2018.
- [116] Diederik P Kingma, Max Welling, et al. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019.
- [117] Martín Abadi et al. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL <https://www.tensorflow.org/>. Software available from [tensorflow.org](https://www.tensorflow.org/).
- [118] Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Proc. Advances in Neural Information Processing Systems*, pages 8024–8035. Curran Associates, Inc., 2019.

- [119] Dawen Liang, Rahul G Krishnan, Matthew D Hoffman, and Tony Jebara. Variational autoencoders for collaborative filtering. In *Proc. International Conference on World Wide Web*, pages 689–698, 2018.
- [120] Ruslan Salakhutdinov, Andriy Mnih, and Geoffrey Hinton. Restricted boltzmann machines for collaborative filtering. In *Proc. International Conference on Machine Learning*, pages 791–798, 2007.
- [121] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. Autorec: Autoencoders meet collaborative filtering. In *Proc. International Conference on World Wide Web*, pages 111–112, 2015.
- [122] Alfredo Nazabal, Pablo M Olmos, Zoubin Ghahramani, and Isabel Valera. Handling incomplete heterogeneous data using vaes. *arXiv preprint arXiv:1807.03653*, 2018.
- [123] Chao Ma, Sebastian Tschiatschek, Konstantina Palla, José Miguel Hernández-Lobato, Sebastian Nowozin, and Cheng Zhang. Eddi: Efficient dynamic discovery of high-value information with partial vae. In *Proc. International Conference on Machine Learning*, 2019.
- [124] Wenbo Gong, Sebastian Tschiatschek, Richard Turner, Sebastian Nowozin, and José Miguel Hernández-Lobato. Icebreaker: Element-wise active information acquisition with bayesian deep latent gaussian model. In *Proc. Advances in Neural Information Processing Systems*, 2019.
- [125] Chao Ma, Wenbo Gong, José Miguel Hernández-Lobato, Noam Koenigstein, Sebastian Nowozin, and Cheng Zhang. Partial vae for hybrid recommender system. In *Proc. NeurIPS Workshop on Bayesian Deep Learning*, 2018.
- [126] Zoubin Ghahramani and Matthew J Beal. Variational inference for bayesian mixtures of factor analysers. In *Proc. Advances in Neural Information Processing Systems*, pages 449–455, 2000.
- [127] Jian-hua Zhao and LH Philip. A note on variational bayesian factor analysis. *Neural Networks*, 22(7):988–997, 2009.
- [128] Mike Wu, Richard L Davis, Benjamin W Domingue, Chris Piech, and Noah Goodman. Variational item response theory: Fast, accurate, and expressive. *arXiv preprint arXiv:2002.00276*, 2020.
- [129] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. International Conference on Learning Representations*, 2015.
- [130] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2009.
- [131] Neil T. Heffernan and Cristina Heffernan. The assistments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*, 24:470–497, 2014.
- [132] J. Stamper, A. Niculescu-Mizil, S. Ritter, G.J. Gordon, and K.R. Koedinger. Algebra i 2006-2007. *Development data set from KDD Cup 2010 Educational Data*

- Mining Challenge*, 2010.
- [133] J. Stamper, A. Niculescu-Mizil, S. Ritter, G.J. Gordon, and K.R. Koedinger. Bridge to algebra 2006-2007. *Development data set from KDD Cup 2010 Educational Data Mining Challenge*, 2010.
 - [134] John C. Stamper and Zachary A. Pardos. The 2010 kdd cup competition dataset: Engaging the machine learning community in predictive learning analytics. In *Journal of Learning Analytics*, 2016.
 - [135] Hemant Ishwaran, J Sunil Rao, et al. Spike and slab variable selection: frequentist and bayesian strategies. *The Annals of Statistics*, 33(2):730–773, 2005.
 - [136] Michalis K Titsias and Miguel Lázaro-Gredilla. Spike and slab variational inference for multi-task and multiple kernel learning. In *Proc. Advances in neural information processing systems*, pages 2339–2347, 2011.
 - [137] Francesco Tonolini, Bjorn Sand Jensen, and Roderick Murray-Smith. Variational sparse coding, July 2019. URL <http://eprints.gla.ac.uk/191553/>.
 - [138] Ruqi Zhang, Chunyuan Li, Jianyi Zhang, Changyou Chen, and Andrew Gordon Wilson. Cyclical stochastic gradient mcmc for bayesian deep learning. *Proc. International Conference on Learning Representations*, 2020.
 - [139] Yi-An Ma, Tianqi Chen, and Emily Fox. A complete recipe for stochastic gradient mcmc. In *Proc. Advances in Neural Information Processing Systems*, pages 2917–2925, 2015.
 - [140] Jiaming Song, Shengjia Zhao, and Stefano Ermon. A-nice-mc: Adversarial training for mcmc. In *Proc. Advances in Neural Information Processing Systems*, pages 5140–5150, 2017.
 - [141] Nando De Freitas, Pedro Højén-Sørensen, Michael I Jordan, and Stuart Russell. Variational mcmc. In *Proc. conference on Uncertainty in Artificial Intelligence*, pages 120–127. Morgan Kaufmann Publishers Inc., 2001.
 - [142] Raza Habib and David Barber. Auxiliary variational mcmc. *Proc. International Conference on Learning Representations*, 2019.
 - [143] Antonio Auffinger and Daniel Fletcher. Equilibrium distributions for t-distributed stochastic neighbour embedding, 2023, 2304.03727. URL <https://arxiv.org/abs/2304.03727>.
 - [144] Aditya Ravuri and Neil D. Lawrence. Towards one model for classical dimensionality reduction: A probabilistic perspective on umap and t-sne, 2024, 2405.17412. URL <https://arxiv.org/abs/2405.17412>.
 - [145] Pavlin G. Poličar and Blaž Zupan. Visualizing high-dimensional temporal data using direction-aware t-sne, 2024, 2403.19040. URL <https://arxiv.org/abs/2403.19040>.
 - [146] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(2579-2605), 2008.
 - [147] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.

- [148] Ben Weinkove. Dimension reduction and the gradient flow of relative entropy, 2024, 2409.16963. URL <https://arxiv.org/abs/2409.16963>.
- [149] Wenjun Liu, Quoc Ngo, and Vu Huy. Several interesting integral inequalities. *Journal of Mathematical Inequalities*, 3, 06 2009.
- [150] Nan Jiang, Yi Gu, and Yexiang Xue. Learning markov random fields for combinatorial structures with sampling through lovász local lemma, 2022.
- [151] T. Yang, C. Studer, R. Baker, N. Hefferenan, and A. S. Lan. Active learning for student affect detection. In *Proc. Educational Data Mining*, 2019.
- [152] Gregory Aist, Barry Kort, Rob Reilly, Jack Mostow, and Rosalind Picard. Experimentally augmenting an intelligent tutoring system with human-supplied capabilities: adding human-provided emotional scaffolding to an automated reading tutor that listens. In *Proc. IEEE International Conference on Multimodal Interfaces*, pages 483–490. IEEE, 2002.
- [153] Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *Proc. International Conference on Machine Learning - Volume, ICML’15*, page 1530–1538. JMLR.org, 2015.
- [154] Durk P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. In *Proc. Advances in Neural Information Processing Systems*, pages 4743–4751, 2016.